



Norme di Progetto

The Bitless (Gruppo 18):

Lorenzo Battistella (2103116), Edoardo De Piccoli (2101055), Davide Facco (2087852),
Anna Marini (2110986), Andrea Menegaldo (2116426), Samuele Vendramin (2111934),
Simone Zecchinato (2113189)

Informazioni documento			
Versione	Data	Stato	Destinatari
2.0.0	2026-08-06	Approvato	The Bitless, prof. Tullio Vardanega, prof. Riccardo Cardin

Contatti: thebitless.swe@gmail.com

Registro delle modifiche					
Versione	Data	Autore	Verificatore	Approvatore	Descrizione
2.0.0	2026-08-06	-	-	S. Zecchinato	Approvazione _G del documento per l'ingresso nella Product Baseline _G
1.6.0	2026-08-04	A. Menegaldo	S. Vendramin	-	Miglioramento dei processi organizzativi: pianificazione _G , coordinamento, miglioramento e formazione passano da descrizione a procedura, con cadenze di presidio e matrice di selezione del canale di comunicazione
1.5.0	2026-08-01	S. Vendramin	D. Facco	-	Miglioramento della gestione della configurazione _G : le indicazioni generali su Git _G diventano regole vincolanti di denominazione dei branch _G , formato dei messaggi di commit _G e checklist di approvazione _G delle pull request _G
1.4.0	2026-07-30	A. Marini	A. Menegaldo	-	Miglioramento dei processi di verifica _G e validazione _G : le tecniche descritte diventano checklist applicabili, con criterio di scelta fra walkthrough e inspection e criteri oggettivi di superamento dei livelli di test _G

Registro delle modifiche					
Versione	Data	Autore	Verificatore	Approvatore	Descrizione
1.3.0	2026-07-28	D. Facco	S. Zecchinato	-	Miglioramento del processo di documentazione: il ciclo di vita dei documenti è riformulato in sette fasi con ruolo e output che abilita la fase successiva; aggiunte la matrice delle responsabilità e le regole <i>Documentation as Code</i>
1.2.0	2026-07-25	S. Zecchinato	L. Battistella	-	Miglioramento del processo di sviluppo _G : le attività descrittive diventano procedure passo-passo per analisi dei requisiti _G , progettazione _G , adozione dei design pattern _G e codifica, con criteri espliciti di rifiuto in verifica _G
1.1.0	2026-07-22	E. De Piccoli	A. Marini	-	Miglioramento del processo di fornitura: le descrizioni delle attività diventano procedure operative con ruolo responsabile, input, azioni e output verificabile, a recepimento del riscontro ricevuto sulla valutazione dei documenti RTB _G
1.0.0	2026-07-08	-	-	S. Vendramin	Approvazione _G del documento per RTB
0.19.0	2026-07-07	A. Marini	S. Vendramin	-	Correzione del valore del BAC e revisione stilistica (terminologia)
0.18.0	2026-07-06	A. Menegaldo	A. Marini	-	Aggiunta delle metriche MPC-MR (Merge Rework) e MPC-IR (Issue Reopening Rate)

Registro delle modifiche					
Versione	Data	Autore	Verificatore	Approvatore	Descrizione
0.17.0	2026-06-04	S. Zecchinato	A. Menegaldo	-	Aggiornamento sezione Metriche di Qualità del prodotto
0.16.0	2026-05-28	S. Vendramin	A. Menegaldo	-	Aggiornamento sezione Metriche di Qualità del prodotto
0.15.1	2026-05-13	A. Marini	S. Vendramin	-	Corretto formato riferimenti
0.15.0	2026-05-11	D. Facco	S. Vendramin	-	Sistemazione Attività di verifica _G (sezione 3.3.2)
0.14.1	2026-05-08	E. De Piccoli	S. Vendramin	-	Correzione errori di battitura
0.14.0	2026-05-07	L. Battistella	S. Vendramin	-	Stesura sezione Metriche per la Qualità del prodotto
0.13.0	2026-05-04	S. Zecchinato	S. Vendramin	-	Stesura sezione Metriche di Qualità del processo
0.12.1	2026-05-03	A. Menegaldo	S. Vendramin	-	Correzioni grammaticali e adeguamento formattazione tabelle
0.12.0	2026-05-02	S. Vendramin	D. Facco	-	Fine sezione Metriche per la qualità
0.11.0	2026-04-30	A. Marini	S. Vendramin	-	Stesura sezione Formazione e inizio Metriche per la qualità
0.10.0	2026-04-28	A. Menegaldo	S. Vendramin	-	Stesura sezione 3.4 Gestione della configurazione
0.9.0	2026-04-26	L. Battistella	E. De Piccoli	-	Stesura sezione Coordinamento e Miglioramento
0.8.1	2026-04-24	S. Zecchinato	E. De Piccoli	-	Aggiornamento sezione Struttura verbali
0.8.0	2026-04-22	S. Zecchinato	E. De Piccoli	-	Stesura Gestione dei Processi
0.7.0	2026-04-18	S. Vendramin	E. De Piccoli	-	Stesura sezione 3.5 Risoluzione dei problemi
0.6.0	2026-04-16	E. De Piccoli	A. Menegaldo	-	Stesura sezione 3.3 Validazione _G
0.5.0	2026-04-15	A. Marini	A. Menegaldo	-	Stesura sezione 3.2 Verifica _G
0.4.0	2026-04-13	S. Zecchinato	A. Menegaldo	-	Stesura sezione 3.1 Documentazione

Registro delle modifiche					
Versione	Data	Autore	Verificatore	Approvatore	Descrizione
0.3.1	2026-04-10	A. Marini	A. Menegaldo	-	Correzione errori ortografici sezione Fornitura
0.3.0	2026-04-08	D. Facco	A. Menegaldo	-	Stesura processo di Sviluppo _G (Processi primari)
0.2.0	2026-04-04	S. Vendramin	A. Menegaldo	-	Stesura processo di Fornitura (Processi primari)
0.1.0	2026-04-01	L. Battistella	A. Menegaldo	-	Struttura del documento e stesura Introduzione

Indice

1	Introduzione	10
1.1	Scopo del documento	10
1.2	Scopo del prodotto	10
1.3	Glossario	11
1.4	Riferimenti	11
1.4.1	Riferimenti normativi	11
1.4.2	Riferimenti informativi	11
2	Processi Primari	11
2.1	Fornitura	12
2.1.1	Processo di Fornitura	12
2.1.2	Attività	12
2.1.3	Documentazione fornita	13
2.1.4	Procedure Operative	15
2.1.4.1	Procedura: comunicazione con la proponente	15
2.1.4.2	Procedura: pianificazione e svolgimento di un incontro SAL . . .	16
2.1.4.3	Procedura: verbalizzazione di una riunione	16
2.1.5	Strumenti	17
2.2	Sviluppo	17
2.2.1	Processo di sviluppo	17
2.2.2	Attività previste	18
2.2.3	Procedure operative	20
2.2.3.1	Procedura: analisi dei requisiti	20
2.2.3.2	Procedura: stesura di un caso d'uso	20
2.2.3.3	Convenzioni: diagrammi dei casi d'uso	22
2.2.3.4	Procedura: definizione di un requisito	22
2.2.3.5	Procedura: progettazione	24
2.2.3.6	Qualità dell'architettura	25
2.2.3.7	Convenzioni: diagrammi UML	26
2.2.3.8	Procedura: adozione di un design pattern	27
2.2.3.9	Procedura: codifica di un'unità software	28
2.2.3.10	Norme di codifica	28
2.2.3.11	Procedura: predisposizione dell'ambiente di esecuzione	30
2.2.4	Strumenti a supporto	30
3	Processi di supporto	31
3.1	Documentazione	31
3.1.1	Processo	31
3.1.1.1	Regole: Documentation as Code	32
3.1.2	Ciclo di vita dei documenti	32
3.1.2.1	Fase 1 — Necessità	33
3.1.2.2	Fase 2 — Pianificazione	33
3.1.2.3	Fase 3 — Redazione	34
3.1.2.4	Fase 4 — Verifica	34
3.1.2.5	Fase 5 — Approvazione	35
3.1.2.6	Fase 6 — Consolidamento	35
3.1.2.7	Fase 7 — Manutenzione	35

3.1.3	Attività previste	35
3.1.3.1	Produzione della documentazione	35
3.1.3.2	Revisione	36
3.1.3.3	Approvazione	36
3.1.3.4	Archiviazione e pubblicazione	36
3.1.3.5	Matrice delle responsabilità documentali	37
3.1.3.6	Redazione dei verbali	37
3.1.3.7	Redazione dell'Analisi dei Requisiti	37
3.1.4	Procedure operative	38
3.1.4.1	Struttura generale dei documenti	38
3.1.4.1.1	Frontespizio	38
3.1.4.1.2	Registro delle modifiche	38
3.1.4.1.3	Indice	38
3.1.4.2	Struttura dei Verbali	39
3.1.4.2.1	Informazioni sulla riunione	39
3.1.4.2.2	Presenze	39
3.1.4.2.3	Ordine del giorno	39
3.1.4.2.4	Discussione e sintesi	39
3.1.4.2.5	Attività da svolgere	40
3.1.4.3	Versionamento	40
3.1.4.3.1	Branch	40
3.1.4.3.2	Stato dei documenti	40
3.1.4.3.3	Nomenclatura	41
3.1.4.3.4	Commit e Pull Request	41
3.1.4.4	Processo di verifica tramite Pull Request	41
3.1.4.5	Processo di approvazione tramite Pull Request	42
3.1.4.6	Acronimi	42
3.1.5	Strumenti di supporto	43
3.1.5.1	L ^A T _E X _G	43
3.1.5.2	GitHub _G	43
3.1.5.3	Visual Studio Code	44
3.1.5.4	Notion	44
3.2	Verifica	44
3.2.1	Processo di verifica	44
3.2.2	Attività di verifica	45
3.2.2.1	Procedura: verifica di un documento	45
3.2.2.2	Procedura: analisi statica	46
3.2.2.3	Procedura: analisi dinamica e testing	47
3.2.2.4	Test di accettazione	47
3.2.2.5	Sequenza delle fasi di test	48
3.2.2.6	Codici identificativi dei test	48
3.2.2.7	Stato dei test	48
3.2.2.8	Integrazione continua	49
3.2.3	Procedure operative	49
3.2.3.1	Procedura di verifica documentale	49
3.2.3.2	Procedura di walkthrough	49
3.2.4	Strumenti di supporto	50
3.2.4.1	Test frontend	50
3.2.4.2	Test backend	50

3.3	Validazione	51
3.3.1	Processo di validazione	51
3.3.2	Attività di validazione	51
3.3.3	Procedure operative	52
3.3.4	Strumenti di supporto	52
3.4	Gestione della configurazione	53
3.4.1	Processo di gestione della configurazione	53
3.4.2	Attività di gestione della configurazione	53
3.4.2.1	Regole per l'identificazione dei Configuration Item	53
3.4.2.2	Regole per il versionamento dei documenti	54
3.4.2.3	Regole per l'uso dei repository	54
3.4.3	Procedure operative	54
3.4.3.1	Regole per la denominazione dei branch	54
3.4.3.2	Regole per i messaggi di commit	55
3.4.3.3	Procedura: apertura di una Pull Request	56
3.4.3.4	Checklist per l'approvazione di una Pull Request	56
3.4.3.5	Controllo di configurazione e Change Request	57
3.4.4	Strumenti di supporto	57
3.4.4.1	Git	58
3.4.4.2	GitHub	58
3.5	Risoluzione dei problemi	58
3.5.1	Processo di risoluzione dei problemi	58
3.5.2	Attività di risoluzione dei problemi	59
3.5.2.1	Gestione dei rischi	59
3.5.2.2	Procedura: segnalazione di un problema	59
3.5.3	Procedure operative	60
3.5.3.1	Codifica dei rischi	60
3.5.3.2	Procedura di segnalazione e gestione dei problemi	60
3.5.4	Strumenti di supporto	61
4	Processi organizzativi	61
4.1	Gestione dei Processi	61
4.1.1	Processo di gestione dei processi	61
4.1.2	Attività di gestione	62
4.1.2.1	Pianificazione	62
4.1.2.2	Assegnazione dei ruoli e responsabilità	63
4.1.2.3	Ticketing e tracciamento delle attività	64
4.1.3	Procedure operative	64
4.1.3.1	Procedura di pianificazione	64
4.1.3.2	Procedura di gestione del ciclo di vita delle issue	65
4.1.4	Strumenti di supporto	65
4.2	Coordinamento	66
4.2.1	Processo di coordinamento	66
4.2.2	Attività di coordinamento	66
4.2.2.1	Riunioni interne	66
4.2.2.2	Riunioni esterne	67
4.2.3	Procedure operative	67
4.2.3.1	Procedura per le riunioni interne	67
4.2.3.2	Procedura per le riunioni esterne	68

4.2.4	Strumenti di supporto	68
4.3	Miglioramento	68
4.3.1	Processo di miglioramento	68
4.3.2	Attività di miglioramento	69
4.3.2.1	Analisi	69
4.3.2.2	Miglioramento	69
4.3.3	Procedure operative	70
4.3.3.1	Procedura di retrospettiva e miglioramento	70
4.3.4	Strumenti di supporto	70
4.4	Formazione	70
4.4.1	Processo di formazione	70
4.4.2	Attività di formazione	71
4.4.2.1	Formazione individuale	71
4.4.2.2	Formazione di gruppo	71
4.4.3	Procedure operative	72
4.4.3.1	Procedura per il passaggio di consegne (rotazione dei ruoli)	72
4.4.3.2	Procedura per le sessioni formative con la proponente	72
4.4.4	Strumenti di supporto	73
5	Metriche e standard per la Qualità	73
5.1	Funzionalità	73
5.2	Affidabilità	74
5.3	Efficienza	74
5.4	Usabilità	74
5.5	Manutenibilità	74
5.6	Portabilità	75
5.7	Nomenclatura delle Metriche	75
5.8	Suddivisione secondo ISO/IEC 12207	75
5.8.0.1	Riferimenti	76
6	Metriche di Qualità del Processo	76
6.1	Processi primari	76
6.1.1	Fornitura	76
6.1.1.1	Earned Value (EV)	76
6.1.1.2	Planned Value (PV)	77
6.1.1.3	Actual Cost (AC)	77
6.1.1.4	Cost Performance Index (CPI)	77
6.1.1.5	Estimate At Completion (EAC)	78
6.1.1.6	Estimate To Complete (ETC)	78
6.1.1.7	Cost Variance (CV)	79
6.1.1.8	Schedule Variance (SV)	79
6.1.1.9	Budget At Completion (BAC)	80
6.2	Processi di supporto	80
6.2.1	Documentazione	80
6.2.1.1	Indice di Gulpease (IG)	80
6.2.1.2	Correttezza Ortografica (CO)	81
6.3	Processi organizzativi	82
6.3.1	Gestione dei processi	82
6.3.1.1	Task Completion Rate (TCR)	82

6.3.1.2	Task Slippage (TS)	82
6.3.1.3	Merge Rework (MR)	83
6.3.1.4	Issue Reopening Rate (IR)	83
6.3.1.5	Percentuale Metriche Soddisfatte (PMS)	84
7	Metriche di Qualità del Prodotto	84
7.1	Funzionalità	84
7.1.1	Copertura Requisiti Obbligatori (MPD-CRO)	84
7.1.2	Copertura Requisiti Desiderabili (MPD-CRD)	85
7.1.3	Copertura Requisiti Opzionali (MPD-CROp)	85
7.1.4	Copertura Funzioni LLM Obbligatorie (MPD-CLLM)	85
7.1.5	Correttezza Rendering Markdown (MPD-CMD)	85
7.1.6	Salvataggio e Recupero Note (MPD-SN)	86
7.2	Affidabilità	86
7.2.1	Data Loss (MPD-DL)	86
7.2.2	Error Handling (MPD-EH)	86
7.2.3	Session Recovery (MPD-SR)	87
7.2.4	Availability Rate (MPD-AR)	87
7.3	Usabilità	87
7.3.1	Learning Time (MPD-LT)	87
7.3.2	Navigation Errors (MPD-NE)	88
7.3.3	UI Consistency (MPD-UI)	88
7.3.4	Feature Understandability (MPD-FU)	88
7.4	Efficienza	89
7.4.1	Response Time Interfaccia (MPD-RT)	89
7.4.2	Markdown Rendering Time (MPD-MRT)	89
7.4.3	LLM Response Time (MPD-LLMRT)	89
7.4.4	Loading Speed (MPD-LS)	90
7.5	Manutenibilità	90
7.5.1	Complessità Ciclomantica (MPD-CYC)	90
7.5.2	Code Smell (MPD-CS)	90
7.5.3	Duplicazione del Codice (MPD-DUP)	91
7.5.4	Modularità (MPD-MOD)	91
7.6	Sicurezza	91
7.6.1	Secret Key Exposure (MPD-SK)	91
7.6.2	Input Validation (MPD-IV)	92

1 Introduzione

1.1 Scopo del documento

Questo documento delinea le modalità operative, denominate Way of Working_G, adottate dal team **The Bitless** durante lo svolgimento del progetto didattico. Il suo scopo è stabilire un protocollo comune di lavoro: una serie di norme che ogni membro del gruppo è tenuto a rispettare. Questo garantisce che lo sviluppo_G proceda in modo fluido, ordinato e con uno standard qualitativo elevato.

Il documento recepisce le direttive del Regolamento del Progetto Didattico e si ispira allo standard internazionale **ISO/IEC 12207:1995**. Tale norma individua tre principali tipologie di processi:

- **Processi Primari:** l'insieme delle attività necessarie alla realizzazione del prodotto, dalla raccolta dei requisiti_G fino alla manutenzione finale;
- **Processi di Supporto:** attività trasversali che garantiscono la solidità del lavoro, come la gestione della configurazione e le attività di test_G;
- **Processi Organizzativi:** compiti dedicati alla gestione delle risorse, alla formazione interna e al costante miglioramento dell'infrastruttura di squadra.

In linea con lo standard, è stato applicato un processo di **personalizzazione (tailoring_G)**, selezionando esclusivamente gli elementi necessari al nostro contesto specifico. Il documento seguirà un approccio **incrementale**, venendo aggiornato e perfezionato man mano che il team acquisisce nuove competenze e feedback_G.

1.2 Scopo del prodotto

Il progetto proposto dal capitolato_G **C6 di Zucchetti S.p.A.** consiste nella realizzazione di **Second Brain**: una piattaforma digitale evoluta basata sullo standard **Markdown_G**.

L'elemento distintivo del sistema_G risiede nell'integrazione_G nativa di modelli di linguaggio estesi (**LLM_G**), finalizzati a consentire il paradigma del **"Distant Writing"**_G. In questo scenario, l'utente assume il ruolo strategico di **progettista del testo** e coordinatore del flusso narrativo, delegando all'intelligenza artificiale l'esecuzione iterativa e l'espansione dei nuclei informativi.

L'apporto degli LLM_G si declina in due aree funzionali:

- **Collaborazione Iterativa e Generativa:** Il sistema_G supporta l'utente nelle operazioni di riscrittura adattiva, sintesi semantica, traduzione multilingue ed estrazione automatica di entità e concetti rilevanti, ottimizzando i tempi di produzione dei contenuti.
- **Analisi Critica:** Attraverso l'applicazione della tecnica dei **"Sei Cappelli per Pensare"** (Edward de Bono), l'assistente virtuale agisce come un revisore analitico capace di esaminare i testi da diverse angolazioni (rischi, benefici, logica e creatività), offrendo un supporto decisionale strutturato che eleva la qualità del prodotto finale.

Il termine ultimo per la consegna del prodotto è fissato per il **30 agosto 2026**, con un budget_G operativo previsto di **12.860,00 €**.

1.3 Glossario

La creazione e lo sviluppo_G di un sistema_G software richiedono una complessa operazione di progettazione e analisi del dominio_G, attività che deve necessariamente precedere la stesura del codice. Il gruppo si impegna a raccogliere e organizzare tali informazioni in modo che siano facilmente accessibili, al fine di favorire la massima asincronia ed efficienza nelle attività di progetto.

La principale fonte di ambiguità nello svolgimento delle attività è rappresentata dall'incomprensione dei termini tecnici o specialistici adottati. A tale scopo, la nomenclatura di riferimento viene raccolta nel *glossario*, un documento incrementale_G volto a definire ogni termine rilevante per il dominio_G del progetto.

Il gruppo si impegna a contrassegnare ogni occorrenza di un termine presente nel glossario mediante l'apposizione di una lettera **G** a pedice, come esemplificato di seguito:

parola_G

Al fine di garantire una corretta comprensione del dominio_G, è fondamentale che ogni membro del gruppo consulti periodicamente il glossario, assicurando così il costante allineamento sulle definizioni e sui concetti chiave.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- **Capitolato C6 - Second Brain:**

<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>

Ultimo accesso: 23 giugno 2026.

1.4.2 Riferimenti informativi

- **Regolamento del Progetto Didattico:**

<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>

Ultimo accesso: 22 maggio 2026.

- **Standard ISO/IEC 12207:1995:**

https://www.math.unipd.it/~tullio/IS-1/2009/Approfondimenti/ISO_12207-1995.pdf

Ultimo accesso: 9 giugno 2026.

- **Glossario v1.0.0:**

<https://thebitless.live/RTB/doc.interna/glossario/glossario.pdf>

Ultimo accesso: 24 giugno 2026.

2 Processi Primari

In conformità allo standard **ISO/IEC 12207:1995**, i processi primari coprono l'intero ciclo di vita del software. Il gruppo **The Bitless** adotta i processi di **Fornitura** (Supply Process) e **Sviluppo** (Development Process).

Le sezioni seguenti sono vincolanti per tutti i membri del gruppo. Ogni procedura specifica il ruolo responsabile, gli input necessari, le azioni da compiere e l'output verificabile atteso. In caso di conflitto fra una procedura e una prassi consolidata, prevale la procedura documentata.

2.1 Fornitura

2.1.1 Processo di Fornitura

Il processo di fornitura regola i rapporti con la proponente_G **Zucchetti S.p.A.** e con la committenza_G didattica, dalla candidatura alla consegna finale.

Ruolo responsabile Responsabile_G di progetto in carica nello sprint_G corrente.

Input Capitolato_G C6; accordi contrattuali; pianificazione_G dello sprint_G corrente.

Output Documentazione di fornitura aggiornata e tracciabile_G; accordi con la proponente_G verbalizzati e approvati_G.

Regole vincolanti del processo:

1. Formalizzare per iscritto ogni accordo assunto con la proponente_G o con la committenza_G: nessun accordo verbale è considerato vincolante finché non è verbalizzato.
2. Tracciare ogni richiesta di modifica ai requisiti_G tramite issue_G dedicata prima di recepirla nei documenti.
3. Sottoporre a verifica_G interna ogni artefatto_G prima dell'invio a soggetti esterni al gruppo.

2.1.2 Attività

Il processo di fornitura si articola nelle sette attività previste dallo standard. Per ciascuna sono indicate le azioni da compiere e l'output atteso.

1. **Initiation** — *Ruolo*: Analista_G.

- Analizzare le richieste avanzate dalla proponente_G (RFP_G);
- Valutare la fattibilità_G tecnica rispetto alle competenze disponibili nel gruppo;
- Redigere l'*Analisi dei Capitolati* con le motivazioni della scelta.

Output: documento *Analisi dei Capitolati* approvato_G.

2. **Preparation of response** — *Ruolo*: Responsabile_G.

- Definire il *Tailoring*_G dello standard, selezionando i soli processi funzionali agli obiettivi del gruppo;
- Redigere la proposta progettuale e la *Lettera di Candidatura*.

Output: *Lettera di Candidatura* inviata alla committenza_G.

3. **Contract** — *Ruolo*: Responsabile_G.

- Concordare deliverable_G, scadenze e criteri di accettazione con docente e tutor aziendale;
- Verbalizzare gli accordi in un *Verbale Esterno* e sottoporlo a firma.

Output: *Verbale Esterno* firmato dalle parti.

4. **Planning** — *Ruolo*: Responsabile_G.

- Selezionare il modello di sviluppo_G e ripartire i ruoli_G per il periodo;
- Allocare le risorse e stimare il monte ore per ruolo;
- Aggiornare l'analisi dei rischi_G nel *Piano di Progetto*.

Output: sezione di pianificazione_G e preventivo_G dello sprint_G nel *Piano di Progetto*.

5. **Execution and control** — *Ruolo:* Responsabile_G, con tutti i ruoli_G operativi.

- Assegnare le issue_G dello sprint_G ai membri secondo la ripartizione dei ruoli_G;
- Monitorare settimanalmente avanzamento, ore consumate e scostamenti_G;
- Attivare le misure correttive previste al superamento delle soglie di rischio_G.

Output: *Kanban board* aggiornata; consuntivo_G orario del periodo.

6. **Review and evaluation** — *Ruolo:* Responsabile_G e Verificatore_G.

- Organizzare gli incontri di revisione con il tutor aziendale;
- Applicare i processi di Verifica_G e Validazione_G agli artefatti_G presentati;
- Registrare i feedback_G ricevuti come issue_G tracciabili_G.

Output: *Verbale Esterno* dell'incontro; issue_G aperte per ogni rilievo ricevuto.

7. **Delivery and completion** — *Ruolo:* Responsabile_G e Amministratore_G.

- Consegnare prodotto e documentazione nella versione approvata_G;
- Fornire supporto durante i test_G di accettazione;
- Archiviare la baseline_G consegnata nel repository_G.

Output: baseline_G rilasciata e tagged nel repository_G.

2.1.3 Documentazione fornita

Redigere e mantenere aggiornati i documenti elencati di seguito. Per ciascun documento la tabella indica il ruolo responsabile della stesura e l'evento che ne impone l'aggiornamento.

Documento	Ruolo responsabile	Aggiornare quando
Lettera di Candidatura	Responsabile _G	Alla candidatura al capitolato _G
Analisi dei Capitolati	Analista _G	Alla selezione del capitolato _G
Dichiarazione degli impegni	Responsabile _G	A ogni variazione del monte ore o dei ruoli _G
Lettera di Presentazione	Responsabile _G	A ogni candidatura a una revisione di avanzamento
Analisi dei Requisiti	Analista _G	A ogni variazione di requisiti _G o casi d'uso _G
Norme di Progetto	Amministratore _G	A ogni modifica del <i>Way of Working</i> _G
Piano di Progetto	Responsabile _G	A ogni apertura e chiusura di sprint _G
Piano di Qualifica	Verificatore _G	A ogni rilevazione di metriche _G o esecuzione di test _G
Specifica Tecnica	Progettista _G	A ogni decisione architeturale assunta
Manuale Utente	Progettista _G	A ogni funzionalità rilasciata all'utente
Glossario	Tutti i ruoli _G	All'introduzione di un termine di dominio _G
Verbalì interni ed esterni	Responsabile _G	Entro 48 ore da ogni riunione

Tabella 2: Documentazione fornita, responsabilità e criteri di aggiornamento

Contenuto minimo obbligatorio di ciascun documento:

- **Lettera di Candidatura:** istanza formale con cui il gruppo manifesta l'interesse per il capitolato_G C6 proposto da **Zucchetti S.p.A.**. Illustra le ragioni della scelta e fornisce le coordinate per accedere ai repository_{GG} ufficiali del progetto.
- **Analisi dei Capitolati:** documento di analisi comparativa che espone le motivazioni tecniche e logistiche che hanno portato alla selezione del progetto **Second Brain** rispetto alle altre proposte disponibili. Contiene un'analisi dettagliata di ciascun capitolato_G, descrivendone l'obiettivo, le tecnologie adottate e le considerazioni emerse durante il confronto nel gruppo.
- **Dichiarazione degli impegni:** documento che ufficializza l'impegno operativo dei membri del team, stabilito in un monte ore produttivo di 93 ore per ciascun componente. Include il piano economico e la strategia di rotazione dei ruoli_{GG} ogni due settimane, volta a garantire un'esperienza formativa completa e un carico di lavoro bilanciato tra i componenti.
- **Lettera di Presentazione:** documento tramite il quale il gruppo intende formalizzare la propria candidatura alle revisioni di avanzamento legate alle *baseline*_G del progetto didattico, ovvero la *Requirements and Technology Baseline* (RTB_G) e la *Product Baseline* (PB_G).

- **Analisi dei Requisiti:** descrive in modo dettagliato le funzionalità e i vincoli del sistema_G **Second Brain** (Capitolato_G C6). L'identificazione delle necessità avviene tramite lo studio dei **Casi d'Uso_G**, corredati da diagrammi illustrativi e matrici di tracciamento_{GG} per assicurare la totale copertura dei requisiti_G concordati.
- **Norme di Progetto:** documento cardine che definisce il *Way of Working_G* del gruppo sulla base dello standard **ISO/IEC 12207:1995**. Al suo interno sono fissate le procedure per ogni attività, le convenzioni per scrivere il codice, gli strumenti da utilizzare e le tecniche per migliorare costantemente la qualità del lavoro prodotto.
- **Piano di Progetto:** elaborato di pianificazione strategica gestito con metodologia incrementale_{GG}. Analizza la ripartizione temporale delle attività, la gestione delle risorse e dei rischi, fornendo un monitoraggio costante dell'avanzamento e del bilancio dei costi_G sostenuti.
- **Piano di Qualifica:** specifica i protocolli di **Verifica** e **Validazione** adottati dal team. Contiene il dettaglio dei test_G eseguiti sul prodotto, le metriche di qualità applicate e i resoconti dei risultati ottenuti per garantire la conformità agli standard prefissati.
- **Glossario:** dizionario condiviso creato per raccogliere e spiegare i termini tecnici legati allo specifico dominio_{GG} del progetto. Il suo scopo è evitare malintesi, assicurando che ogni parola abbia un unico significato per tutti i membri; i termini presenti in questa lista sono contrassegnati nei documenti dal pedice "G".
- **Verbalì (Interni ed Esterni):** documenti che contengono il riassunto delle discussioni e l'elenco delle decisioni prese durante le riunioni. I verbalì interni tracciano l'evoluzione del progetto tra i componenti di **The Bitless**, mentre quelli esterni cristallizzano feedback_G e scadenze concordate con il committente_{GG}.

2.1.4 Procedure Operative

2.1.4.1 Procedura: comunicazione con la proponente

Ruolo responsabile Responsabile_G di progetto.

Input Quesito o comunicazione da inoltrare, concordato internamente al gruppo.

Output Messaggio inviato dall'indirizzo ufficiale e archiviato; eventuale risposta registrata come issue_G.

Regole ferree:

1. Inviare ogni comunicazione formale **esclusivamente** dall'indirizzo email dedicato fornito da Zucchetti. È vietato utilizzare canali personali per comunicazioni di progetto.
2. Concordare il contenuto del messaggio con il gruppo prima dell'invio: nessun membro contatta la proponente_G a titolo individuale.
3. Mantenere un unico thread per argomento, così da preservare la storicità delle decisioni.
4. Registrare come issue_G ogni risposta che comporti una modifica a requisiti_G, scadenze o deliverable_G.

2.1.4.2 Procedura: pianificazione e svolgimento di un incontro SAL

Ruolo responsabile Responsabile_G di progetto.

Input Necessità di validare una milestone_G o di risolvere una criticità; stato di avanzamento aggiornato.

Output *Verbale Esterno* redatto, verificato_G e firmato; issue_G aperte per ogni azione concordata.

Passi da eseguire:

1. Verificare che sussista una delle condizioni che giustificano l'incontro: validazione_G di una milestone_G, criticità bloccante, o richiesta esplicita della proponente_G.
2. Inviare la richiesta di incontro con **almeno 48 ore di preavviso**, indicando ordine del giorno e durata stimata.
3. Redigere, prima dell'incontro, una sintesi scritta contenente:
 - obiettivi raggiunti dall'ultimo incontro;
 - difficoltà tecniche o organizzative riscontrate;
 - elenco puntuale dei quesiti da porre.
4. Distribuire la sintesi a tutti i membri almeno 24 ore prima dell'incontro.
5. Designare, prima dell'inizio, il membro incaricato della verbalizzazione.
6. Svolgere l'incontro seguendo l'ordine del giorno; annotare ogni decisione nel momento in cui viene assunta.
7. Applicare la procedura di verbalizzazione descritta di seguito.

2.1.4.3 Procedura: verbalizzazione di una riunione

Ruolo responsabile Membro designato come estensore; Verificatore_G per la revisione.

Input Appunti raccolti durante la riunione; template_G del verbale_G.

Output Verbale_G approvato_G, versionato_G e archiviato nel repository_G.

Passi da eseguire:

1. Redigere il verbale_G **entro 48 ore** dal termine della riunione, utilizzando il template_G ufficiale.
2. Includere obbligatoriamente: data e ora, elenco dei partecipanti con stato di presenza, ordine del giorno, discussione per punto, decisioni assunte e attività assegnate con relativo incaricato.
3. Sottoporre il verbale_G a *peer-review* da parte di un membro **diverso dall'estensore**, che ne verifica_G correttezza dei contenuti, completezza e forma.
4. Correggere i rilievi emersi dalla revisione e sottoporre nuovamente il documento fino ad assenza di rilievi.

5. Per i verbali esterni: inviare il documento alla controparte e richiederne la firma.
6. Archiviare il verbale_G nel repository_G secondo le convenzioni di denominazione definite nella Sezione 2.1.4.3.

Criterio di accettazione: il verbale_G è considerato completo quando ogni decisione assunta è associata a un incaricato e, ove pertinente, a una issue_G aperta.

2.1.5 Strumenti

Utilizzare gli strumenti elencati secondo la destinazione d'uso indicata. È vietato impiegare uno strumento per finalità diverse da quelle assegnate.

Strumento	Usare per	Non usare per
GitHub Issues	Tracciare unità di lavoro, assegnare responsabilità, segnalare anomalie	Discussioni informali
GitHub Projects	Pianificare e monitorare l'avanzamento tramite <i>Kanban board</i>	Archiviare documenti
Discord	Riunioni sincrone interne al gruppo	Comunicazioni verso l'esterno
WhatsApp	Comunicazioni rapide asincrone fra membri	Decisioni di progetto non verbalizzate
Google Mail	Corrispondenza formale con proponente _G e committenza _G	Comunicazioni interne
Google Meet	Conferenze da remoto con la proponente _G	Riunioni interne

Tabella 3: Strumenti a supporto della fornitura e relativa destinazione d'uso

Regole ferree sull'uso degli strumenti:

1. Aprire una issue_G per ogni unità di lavoro prima di iniziarne l'esecuzione: nessuna attività può essere svolta senza una issue_G che la tracci.
2. Aggiornare lo stato della issue_G sulla *Kanban board* entro la giornata in cui l'attività cambia stato.
3. Riportare su GitHub ogni decisione presa su Discord o WhatsApp che incida su requisiti_G, scadenze o assegnazioni.

2.2 Sviluppo

2.2.1 Processo di sviluppo

Il processo di sviluppo_G copre analisi, progettazione, codifica, integrazione_G, test_G, installazione e accettazione.

Ruoli responsabili Analista_G (analisi), Progettista_G (progettazione), Programmatore_G (codifica e integrazione_G), Verificatore_G (test_G e collaudo_G).

Input Requisiti_G concordati con la proponente_G; *Analisi dei Requisiti* approvata_G.

Output Prodotto software conforme ai requisiti_G, verificato_G e collaudato_G; *Specifica Tecnica* aggiornata.

Vincoli non derogabili:

1. Non avviare la codifica di un'unità software_G prima che ne esista la progettazione_G di dettaglio approvata_G.
2. Non integrare nel branch_G principale codice privo dei relativi test_G unitari.
3. Non modificare un requisito_G senza aggiornare contestualmente i casi d'uso_G e le tabelle di tracciamento_G collegate.

2.2.2 Attività previste

Il processo si articola nelle tredici attività previste dalla subclause 5.3 dello standard. Per ciascuna sono indicati ruolo, input e output verificabile.

Attività	Input	Ruolo	Output verificabile
Process implementation	Capitolato _G , vincoli _G tecnologici	Responsabile _G	Piani operativi e stack tecnologico documentati
System requirements analysis	RFP _G , verbali _G esterni	Analista _G	Requisiti _G di sistema _G tracciabili _G
System architectural design	Requisiti _G di sistema _G	Progettista _G	Architettura _G ad alto livello
Software requirements analysis	Architettura _G di sistema _G	Analista _G	Requisiti _G software e di qualificazione _G
Software architectural design	Requisiti _G software	Progettista _G	Struttura dei moduli _G e schema del database _G
Software detailed design	Architettura _G software	Progettista _G	Diagrammi UML _G di dettaglio
Software coding and testing	Progettazione _G di dettaglio	Programmatore _G	Codice sorgente con test _G unitari superati
Software integration	Unità software _G testate	Programmatore _G	Moduli _G integrati e test _G di integrazione _G superati
Software qualification testing	Sistema _G software integrato	Verificatore _G	Rapporto di conformità ai requisiti _G software
System integration	Software e dipendenze esterne	Programmatore _G	Sistema _G completo assemblato
System qualification testing	Sistema _G integrato	Verificatore _G	Rapporto di collaudo _G finale
Software installation	Sistema _G collaudato _G	Amministratore _G	Deployment funzionante nell'ambiente target
Software acceptance support	Prodotto installato	Responsabile _G	Verbale _G di accettazione

Tabella 4: Attività del processo di sviluppo: ruoli, input e output

2.2.3 Procedure operative

2.2.3.1 Procedura: analisi dei requisiti

Ruolo responsabile Analista_G.

Input Capitolato_G C6; verballi_G esterni con la proponente_G; risposte ricevute via email.

Output Documento *Analisi dei Requisiti* verificato_G e approvato_G, con tabelle di tracciamento_G complete.

Passi da eseguire:

1. Analizzare il capitolato_G e i verballi_G per individuare le necessità della proponente_G.
2. Formulare per iscritto i quesiti sui punti ambigui e inoltrarli secondo la procedura di comunicazione.
3. Redigere i casi d'uso_G applicando la procedura descritta nella Sezione 2.2.3.2.
4. Derivare i requisiti_G dai casi d'uso_G applicando la procedura della Sezione 2.2.3.4.
5. Compilare la tabella di tracciamento_G requisito–fonte e la tabella requisito–caso d'uso.
6. Sottoporre il documento al Verificatore_G.
7. Correggere i rilievi e ripetere la verifica_G fino ad assenza di rilievi.

Criteri di accettazione: il documento è accettabile solo se **ogni** requisito_G è tracciato_G a una fonte, **ogni** requisito_G funzionale è associato ad almeno un caso d'uso_G e **nessun** requisito_G contiene formulazioni ambigue o non misurabili.

Struttura obbligatoria del documento *Analisi dei Requisiti*:

1. **Introduzione:** finalità, scopo e riferimenti del documento;
2. **Descrizione del prodotto:** prospettiva, obiettivi, funzionalità principali, utenza di riferimento;
3. **Casi d'uso:** attori coinvolti, elenco dei casi d'uso_G in formato testuale strutturato, diagrammi UML_G;
4. **Requisiti:** Requisiti Funzionali_G, qualitativi e di vincolo_G;
5. **Tracciamento:** tabelle requisito–fonte e requisito–caso d'uso.

2.2.3.2 Procedura: stesura di un caso d'uso

Ruolo responsabile Analista_G.

Input Funzionalità individuata nel capitolato_G o in un verbale_G; elenco degli attori già definiti.

Output Caso d'uso_G testuale completo e relativo diagramma UML_G.

Passi da eseguire:

1. Assegnare l'identificativo secondo il formato definito di seguito, verificando che non sia già in uso.

2. Individuare attore principale ed eventuali attori secondari.
3. Definire precondizioni e postcondizioni in forma verificabile.
4. Descrivere lo scenario principale come sequenza numerata di passi.
5. Individuare scenari alternativi, estensioni, inclusioni e generalizzazioni.
6. Produrre il diagramma UML_G corrispondente secondo le convenzioni definite.
7. Associare il caso d'uso G ai requisiti G da esso derivati nella tabella di tracciamento G .

Compilare obbligatoriamente tutti i campi seguenti:

1. **Identificativo**, nel formato: **UC[Primario].[Secondario] – [Titolo]** dove:
 - **UC**: acronimo di *Use Case*;
 - **[Primario]**: ID_G numerico progressivo del caso d'uso principale;
 - **[Secondario]**: ID_G numerico del sottocaso d'uso, presente esclusivamente per casi d'uso G dipendenti dal caso radice;
 - **[Titolo]**: denominazione breve ed esplicativa della funzionalità.
2. **Attore principale**: entità esterna che interagisce attivamente con il sistema_{GG} per soddisfare una propria necessità;
3. **Attore secondario**: eventuale entità esterna che non interagisce direttamente, ma la cui partecipazione è necessaria al sistema_{GG} per soddisfare il bisogno dell'attore principale;
4. **Precondizioni**: stato in cui deve trovarsi il sistema_{GG} affinché la funzionalità sia disponibile all'attore;
5. **Postcondizioni**: stato in cui si trova il sistema_{GG} al termine dell'esecuzione corretta dello scenario principale;
6. **Scenario principale**: sequenza ordinata di eventi che si verificano quando l'attore interagisce con il sistema_{GG} per raggiungere l'obiettivo del caso d'uso;
7. **Scenari alternativi**: sequenze di eventi che si verificano al verificarsi di condizioni specifiche, rappresentando deviazioni dal percorso principale, raggiungendo una postcondizione, ma non quella postcondizioni attese;
8. **Estensioni**: scenari alternativi che, al verificarsi di condizioni specifiche, deviano il flusso dal percorso principale senza giungere alle postcondizioni attese;
9. **Inclusioni**: scenari che rappresentano funzionalità comuni a più casi d'uso_G, richiamati obbligatoriamente all'interno del flusso principale per evitare duplicazioni;
10. **Generalizzazioni**: scenari che rappresentano funzionalità condivise tra più casi d'uso_G, in cui un caso d'uso figlio eredita il comportamento da un caso d'uso padre, consentendo di modellare varianti specifiche senza duplicare il flusso principale;
11. **Trigger**: evento esterno che innesca l'esecuzione del caso d'uso, come un'azione dell'utente o una condizione di sistema_G.

2.2.3.3 Convenzioni: diagrammi dei casi d'uso

Disegnare i diagrammi dei casi d'uso_G rispettando le convenzioni seguenti. Non introdurre dettagli implementativi nei diagrammi.

Rappresentare gli elementi come segue:

- **Attori:** entità esterne al sistema_{GG} che interagiscono con esso (utenti umani, altri sistemi software_G o componenti esterne). Sono rappresentati come *stickman* posizionati all'esterno del rettangolo che delimita il sistema_{GG}.
- **Casi d'uso:** funzionalità offerte dal sistema_{GG} con cui un attore può interagire. Ogni caso d'uso è rappresentato da un'ellisse contenente il proprio identificativo e un titolo esplicativo.
- **Sistema:** rappresentato da un rettangolo con titolo identificativo. Al suo interno sono collocati i casi d'uso_G; all'esterno, gli attori.

Relazioni tra attori e casi d'uso:

- **Associazione:** linea continua che collega un attore a un caso d'uso, indicando il coinvolgimento dell'attore nell'interazione descritta.

Relazioni tra attori:

- **Generalizzazione tra attori:** relazione di ereditarietà in cui un attore figlio eredita comportamenti e caratteristiche dall'attore padre. Rappresentata con una linea continua e freccia vuota dall'attore figlio all'attore padre.

Relazioni tra casi d'uso:

- **Inclusione (⟨include⟩):** indica che un caso d'uso includente incorpora obbligatoriamente l'esecuzione di un caso d'uso incluso. La relazione è rappresentata da una freccia tratteggiata orientata dal caso d'uso includente verso quello incluso. È utile per modellare comportamenti comuni a più casi d'uso_G, evitando duplicazioni.
- **Estensione (⟨extend⟩):** indica che un caso d'uso estendente può ampliare il comportamento di un caso d'uso base al verificarsi di condizioni specifiche. Rappresentata da una freccia tratteggiata orientata dal caso d'uso estendente verso quello esteso. Consente di gestire scenari alternativi senza appesantire il flusso principale.
- **Generalizzazione tra casi d'uso:** relazione di ereditarietà in cui un caso d'uso specifico eredita il comportamento da un caso d'uso più generico. Rappresentata con una linea continua e freccia vuota dal caso d'uso figlio al caso d'uso padre.

2.2.3.4 Procedura: definizione di un requisito

Ruolo responsabile Analista_G.

Input Caso d'uso_G già redatto, oppure indicazione esplicita da capitolato_G o verbale_G.

Output Requisito_G classificato, identificato univocamente e tracciato_G a fonte e casi d'uso_G.

Passi da eseguire:

1. Classificare il requisito_G in una delle tre tipologie:

- **Funzionale (RF)**: comportamento atteso del sistema_G in risposta a input_G o condizioni;
 - **Qualitativo (RQ)**: criterio di prestazione, qualità o usabilità;
 - **Di vincolo (RV)**: limitazione tecnologica, normativa o architettuale.
2. Assegnare l'identificativo progressivo secondo il formato definito di seguito.
 3. Attribuire la priorità fra obbligatorio, desiderabile e opzionale.
 4. Formulare la descrizione in modo **atomico** (un solo comportamento per requisito_G) e **misurabile** (con soglia o criterio oggettivo per i requisiti_G qualitativi).
 5. Indicare la fonte e i casi d'uso_G associati.
 6. Aggiornare le tabelle di tracciamento_G.

Rifiutare in verifica_G ogni requisito_G che presenti almeno uno di questi difetti: descrizione che copre più comportamenti distinti; formulazione non misurabile (es. “il sistema deve essere veloce”); assenza di fonte; assenza di caso d'uso_G associato per i requisiti_G funzionali.

Compilare obbligatoriamente tutti i campi seguenti:

1. **Identificativo**, nel formato: **R[Tipologia][Codice]** dove:
 - **Tipologia**:
 - **RF**: Requisito Funzionale;
 - **RQ**: Requisito Qualitativo;
 - **RV**: Requisito di Vincolo_G.
 - **Codice**: identificativo numerico progressivo univoco all'interno della tipologia di appartenenza.
2. **Priorità**:
 - **Obbligatorio**: requisito irrinunciabile per uno o più stakeholder_G; il mancato soddisfacimento pregiudica il rilascio del prodotto;
 - **Desiderabile**: non strettamente necessario al rilascio, ma apporta valore aggiunto significativo;
 - **Opzionale**: requisito accessorio, implementabile subordinatamente alla disponibilità di risorse o negoziabile in fasi successive.
3. **Descrizione**: narrazione chiara, univoca e dettagliata del comportamento o della caratteristica che il software_G deve possedere, priva di ambiguità interpretative;
4. **Fonte**: origine del requisito, ad esempio: Capitolato_G, Verbale interno, Verbale esterno;
5. **Casi d'uso associati**: elenco degli identificativi UC correlati al requisito.

2.2.3.5 Procedura: progettazione

Ruolo responsabile Progettista_G.

Input *Analisi dei Requisiti* approvata_G; vincoli_G tecnologici; esito del *Proof of Concept* (PoC_G) ove disponibile.

Output *Specifica Tecnica* aggiornata con architettura_G, pattern_G adottati e diagrammi UML_G di dettaglio.

Passi da eseguire, in questo ordine:

1. Svolgere l'analisi tecnologica preliminare e documentare le alternative valutate.
2. Definire l'architettura_G ad alto livello: macro-componenti, responsabilità di ciascuno, interfacce fra essi.
3. Verificare l'architettura_G rispetto agli attributi di qualità elencati più avanti; correggere prima di procedere.
4. Realizzare o aggiornare il PoC_G per validare le scelte a maggior rischio_G tecnico.
5. Scomporre le componenti in unità software_G elementari, ciascuna codificabile in modo indipendente.
6. Produrre i diagrammi UML_G di dettaglio secondo le convenzioni definite più avanti.
7. Documentare ogni decisione architetturale nella *Specifica Tecnica*, motivandone il *perché*.
8. Sottoporre la *Specifica Tecnica* al Verificatore_G prima di rilasciarla ai Programmatori_G.

Regola ferrea: non avviare la codifica prima che l'architettura_G sia definita e verificata_G. Il gruppo persegue la **correttezza per costruzione** e non la correttezza per correzione.

Criterio di completezza di un artefatto_G di progettazione: un artefatto_G è consegnabile ai Programmatori_G solo se specifica, per ogni interfaccia fra moduli_G, la firma dei metodi, i tipi dei parametri e il comportamento atteso in caso di errore. In assenza di tali elementi il Verificatore_G respinge l'artefatto_G.

Contenuto obbligatorio della *Specifica Tecnica*:

- **Tecnologie utilizzate:** specifica delle tecnologie, dei framework_{GG} e delle librerie_G di terze parti integrate nel sistema_{GG}, con motivazione delle scelte effettuate;
- **Architettura logica:** definizione dei componenti del sistema_{GG}, dei loro ruoli, delle connessioni e delle modalità di interazione;
- **Architettura di deployment:** descrizione di come le componenti architetturali vengono allocate e distribuite nell'ambiente di esecuzione;
- **Pattern architetturali e design pattern_G:** descrizione dei pattern adottati, con motivazione delle scelte e rappresentazione grafica del loro funzionamento nel contesto del progetto;
- **Schema del database_G:** modello logico delle entità, con attributi, chiavi primarie, chiavi esterne e vincoli di integrità referenziale;

- **Vincoli e linee guida:** restrizioni e regole progettuali da rispettare durante lo sviluppo_G;
- **Requisiti tecnici:** specifiche prestazionali, di sicurezza, di scalabilità e di compatibilità che il sistema_{GG} deve soddisfare.

2.2.3.6 Qualità dell'architettura

Verificare l'architettura_G prodotta rispetto a ciascuno degli attributi seguenti prima di rilasciarla ai Programmatori_G. Registrare l'esito della verifica_G nel *Piano di Qualifica*; in caso di esito negativo su anche un solo attributo, correggere l'architettura_G e ripetere la verifica_G.

- **Sufficienza:** soddisfazione di tutti i Requisiti Funzionali_G e non funzionali definiti, senza sovradimensionamento né sottodimensionamento;
- **Comprensibilità:** chiarezza strutturale tale da permettere a sviluppatori e stakeholder_{GG} di comprendere il funzionamento del sistema_{GG} senza ambiguità;
- **Modularità:** suddivisione in moduli o componenti chiaramente definiti, con separazione netta delle responsabilità, per facilitare manutenzione e sviluppo_G parallelo;
- **Robustezza:** capacità di gestire situazioni anomale o errate senza interruzioni gravi o perdite di dati;
- **Flessibilità:** facilità di adattamento o estensione del sistema_{GG} a fronte di nuovi requisiti_G, senza richiedere modifiche strutturali radicali;
- **Riusabilità:** capacità di riutilizzare componenti in contesti diversi, riducendo lo sforzo complessivo di sviluppo_G;
- **Efficienza:** utilizzo ottimale delle risorse disponibili (memoria, CPU, rete) per l'esecuzione delle funzionalità nel minor tempo possibile;
- **Affidabilità:** capacità del sistema_{GG} di svolgere correttamente le proprie funzioni in modo coerente e prevedibile nel tempo;
- **Sicurezza rispetto a malfunzionamenti (Safety):** prevenzione di danni a persone, beni o dati causati da malfunzionamenti del software_G;
- **Sicurezza rispetto a intrusioni (Security):** protezione del sistema_{GG} da accessi non autorizzati, manipolazioni e intrusioni esterne;
- **Semplicità:** mantenimento dell'architettura_G il più semplice possibile senza compromettere funzionalità o efficacia;
- **Incapsulazione:** occultamento dei dettagli implementativi all'esterno di ciascuna componente, con accesso consentito esclusivamente tramite interfacce definite;
- **Coesione:** elevata correlazione tra gli elementi interni di ciascun modulo, che devono collaborare verso un obiettivo comune ben definito;
- **Basso accoppiamento:** minimizzazione delle dipendenze tra moduli distinti, per migliorare manutenibilità_G, testabilità e flessibilità del sistema_{GG}.

2.2.3.7 Convenzioni: diagrammi UML

Documentare ogni scelta progettuale con i diagrammi UML_G previsti. Produrre obbligatoriamente:

1. un **diagramma delle classi** per ogni package_G del sistema_G;
2. un **diagramma di sequenza** per ogni caso d'uso_G di priorità obbligatoria;
3. un **diagramma dei package** che rappresenti la suddivisione in livelli dell'architettura_G.

Aggiornare i diagrammi contestualmente a ogni modifica del codice che ne alteri la struttura: un diagramma non allineato al codice costituisce un difetto_G rilevabile in verifica_G.

Diagrammi delle classi. Rappresentare ciascuna classe come un rettangolo suddiviso in tre sezioni, compilate secondo le regole seguenti:

1. **Nome della classe:** identificativo in grassetto, secondo la convenzione *PascalCase*, che deve esplicitare chiaramente la responsabilità della classe. Se la classe è astratta, il nome è scritto in corsivo;
2. **Attributi:** ciascun attributo occupa una riga separata, nel formato: **Visibilità Nome: Tipo [Molteplicità] = ValoreDefault** dove:
 - **Visibilità:** - privata, + pubblica, # protetta, ~ di package;
 - **Nome:** identificativo rappresentativo, in notazione *camelCase*; se costante, in *UPPER_SNAKE_CASE*;
 - **Molteplicità:** per sequenze di elementi, espressa come **tipo[n]** o **tipo[*]** se la cardinalità non è nota a priori;
 - **Valore di default:** opzionale, specificato ove applicabile.
3. **Firme dei metodi:** ciascun metodo occupa una riga separata, nel formato: **Visibilità Nome(Parametri): TipoRitorno** dove:
 - **Visibilità:** segue la convenzione definita per gli attributi;
 - **Nome:** identificativo univoco in *PascalCase*, descrittivo dell'obiettivo del metodo;
 - **Parametri:** da 0 a *n*, separati da virgola, con la stessa notazione degli attributi;
 - **Tipo di ritorno:** tipo del valore restituito.

Convenzioni aggiuntive sui metodi:

- I metodi *getter*, *setter* e i costruttori non vengono inclusi tra i metodi rappresentati;
- I metodi astratti sono scritti in corsivo;
- I metodi statici sono sottolineati;
- L'assenza di attributi o metodi determina la visualizzazione della sezione corrispondente come campo vuoto.

Relazioni tra le classi.

- **Dipendenza:** la classe A utilizza servizi o membri della classe B. Rappresentata con una freccia tratteggiata da A verso B. Indica il minor grado di accoppiamento: un cambiamento in B può influenzare A, ma non viceversa;

- **Associazione:** la classe A contiene campi o istanze della classe B. Rappresentata con una linea continua orientata. Le molteplicità sono espresse agli estremi della freccia: 0..1, 0..*, 1, *, n;
- **Aggregazione:** relazione “parte di” (*part-of*) in cui una classe è composta da altre classi, che tuttavia possono esistere indipendentemente dalla classe principale. Rappresentata con una freccia a diamante vuoto sul lato dell’aggregante;
- **Composizione:** forma più forte di aggregazione in cui le parti non possono esistere indipendentemente dalla classe principale. Rappresentata con un diamante pieno sul lato del componente principale;
- **Generalizzazione:** relazione di ereditarietà tra classi. Rappresentata con una linea continua e freccia vuota dalla classe figlia alla classe padre.

Diagrammi di sequenza. I diagrammi di sequenza rappresentano la dimensione dinamica del sistema_G, descrivendo le interazioni tra oggetti in funzione del tempo per i casi d’uso_G principali. Consentono di verificare la correttezza dei flussi di comunicazione inter-modulo prima della fase di codifica.

2.2.3.8 Procedura: adozione di un design pattern

Ruolo responsabile Progettista_G.

Input Problema di progettazione ricorrente individuato nell’architettura_G.

Output Pattern_G documentato nella *Specifica Tecnica* con motivazione, schema e alternative scartate.

Passi da eseguire:

1. Formulare per iscritto il problema di progettazione da risolvere.
2. Individuare almeno **due** pattern_G candidati applicabili al problema.
3. Confrontare i candidati rispetto agli attributi di qualità dell’architettura_G e alle tecnologie adottate.
4. Assumere la decisione entro il limite di durata prestabilito per la sessione; allo scadere, decidere sulla base delle alternative già istruite.
5. Documentare nella *Specifica Tecnica*, per il pattern_G scelto:
 - la **rappresentazione grafica** del suo funzionamento nel contesto del progetto;
 - la **spiegazione testuale** della logica e del problema risolto;
 - la **motivazione della scelta** e le alternative scartate, con il relativo *perché*.

Regola ferrea: non adottare un pattern_G privo di documentazione della motivazione. Un pattern_G documentato senza alternative scartate è respinto in verifica_G.

2.2.3.9 Procedura: codifica di un'unità software

Ruolo responsabile Programmatore_G.

Input Progettazione_G di dettaglio approvata_G dell'unità; issue_G assegnata; branch_G creato secondo le regole di denominazione.

Output Codice sorgente conforme alle norme di codifica, con test_G unitari superati e copertura_G minima raggiunta, integrato tramite pull request_G approvata_G.

Passi da eseguire, in questo ordine:

1. Verificare che la progettazione_G di dettaglio dell'unità sia disponibile e approvata_G. In caso contrario, sospendere e segnalare al Progettista_G.
2. Creare il branch_G dedicato secondo le regole di denominazione definite nella Sezione 3.4.
3. Implementare l'unità rispettando le norme di codifica elencate di seguito.
4. Scrivere i test_G unitari dell'unità fino al raggiungimento della copertura_G minima dell'**80%** dei rami logici.
5. Eseguire in locale formattatore, linter e suite di test_G; correggere ogni segnalazione prima di procedere.
6. Aggiornare la documentazione inline e i diagrammi UML_G se la struttura è cambiata.
7. Aprire la pull request_G e applicare la procedura definita nella Sezione 3.4.

Condizione di completamento: l'unità è completa solo quando la pipeline di integrazione continua_G è verde, la copertura_G è pari o superiore alla soglia e la pull request_G ha ricevuto l'approvazione_G del Verificatore_G.

2.2.3.10 Norme di codifica

Applicare le norme seguenti a ogni riga di codice prodotta. Sono formalizzate a partire da “*Clean Code*” di Robert C. Martin e dalle best practice_G dei linguaggi adottati. Il Verificatore_G respinge la pull request_G in caso di violazione di una qualsiasi di esse.

- **Nomi significativi:** utilizzare nomi che riflettano il significato e lo scopo delle variabili, funzioni e classi. Evitare abbreviazioni ambigue o identificatori generici come `tmp`, `data`, `obj`. I nomi devono essere autoesplicativi e non richiedere commenti aggiuntivi per essere compresi;
- **Lingua:** identificatori, commenti e documentazione inline devono essere redatti in lingua inglese, per favorire uniformità e compatibilità con strumenti di analisi statica internazionali;
- **Convenzioni di nomenclatura:**
 - *camelCase* per variabili e funzioni in React/JavaScript_G;
 - *snake_case* per variabili, funzioni e nomi di moduli in Python_G;
 - *PascalCase* per classi in Python_G e componenti React_G;

- *UPPER_SNAKE_CASE* per costanti globali, variabili d'ambiente e costanti di modulo;
 - *kebab-case* per i nomi dei file dei componenti React_G (es. `my-component.jsx`).
 - **Indentazione e formattazione:** utilizzo di 4 spazi per l'indentazione in Python_G , 2 spazi per JavaScript/ React_G , in conformità con PEP 8 e le best practice React. La formattazione automatica del codice è delegata agli strumenti **black** (per Python_G) e **Prettier** (per React_G), la cui configurazione è condivisa nel repository $_G$;
 - **Lunghezza delle righe:** le righe di codice non devono superare i **120 caratteri**. Il rispetto di tale limite favorisce la leggibilità su schermi di diverse dimensioni e la visualizzazione affiancata di più file;
 - **Lunghezza e responsabilità dei metodi:** ogni funzione o metodo deve svolgere una **singola responsabilità ben definita**, in conformità al principio *Single Responsibility Principle* (SRP_G) dei principi SOLID_G . La complessità ciclomatica di ogni funzione non deve superare il valore di **10**; qualora superata, il codice deve essere refactorizzato in unità più piccole. Questo favorisce:
 - Chiarezza e leggibilità;
 - Manutenibilità $_G$ e comprensibilità;
 - Testabilità: funzioni brevi sono più semplici da isolare nei Test Unitari $_{GG}$.
 - **Documentazione inline:** ogni funzione pubblica, classe e modulo deve essere documentato con commenti nel formato standard del linguaggio adottato:
 - **docstring** in formato Google style per i moduli, le classi e le funzioni in Python_G ;
 - **JSDoc** per le funzioni e i componenti React_G .
- I commenti devono descrivere il *perché* di una scelta implementativa non ovvia, non il *cosa* fa il codice, che deve essere autoesplicativo;
- **Gestione degli errori:** in Python_G , le eccezioni vanno gestite esplicitamente con blocchi `try-except`, evitando la cattura generica di **Exception** priva di un'adeguata gestione o registrazione nel log. In React_G , le eccezioni devono essere intercettate tramite componenti Error Boundary e le operazioni asincrone gestite con `.catch()` o blocchi `try-catch` all'interno di funzioni `async`;
 - **Test unitari $_G$:** ogni unità software $_{GG}$ prodotta deve essere accompagnata dai relativi Test Unitari $_{GG}$. La copertura minima dei rami logici richiesta è dell'**80%**. I test $_G$ devono essere indipendenti, ripetibili e privi di dipendenze da stato globale o da risorse esterne non mockate. Per Python si utilizza `pytest`, per React si impiegano `Jest` e `React Testing Library`;
 - **Assenza di codice commentato:** il codice non più utilizzato deve essere eliminato e non lasciato commentato nel sorgente. Il versionamento tramite `Git_{GG}` garantisce la possibilità di recuperare versioni precedenti senza necessità di conservare codice morto.

2.2.3.11 Procedura: predisposizione dell'ambiente di esecuzione

Ruolo responsabile Amministratore_G per la configurazione_G; ogni Programmatore_G per l'avvio locale.

Input Repository_G clonato; Docker e Docker Compose installati.

Output Ambiente locale funzionante e allineato all'ambiente di produzione.

Passi da eseguire al primo avvio:

1. Clonare il repository_G e posizionarsi sul branch_G principale.
2. Copiare il file `.env.example` in `.env` e valorizzare le variabili d'ambiente richieste.
3. Eseguire `docker compose up` per avviare i container di backend Python e del server Nginx che serve il frontend_G React compilato.
4. Verificare che entrambi i servizi rispondano prima di iniziare a sviluppare.

Regole ferree sull'ambiente:

1. Dichiarare ogni nuova dipendenza in `requirements.txt` o `package.json` nello stesso commit_G che ne introduce l'uso.
2. Iniettare le variabili d'ambiente a runtime: è **vietato** includere segreti, chiavi API_G o credenziali nelle immagini o nel codice versionato_G.
3. Non modificare la configurazione_G dei container senza aprire una issue_G e ottenere l'approvazione_G dell'Amministratore_G.
4. Mantenere l'ambiente di sviluppo_G allineato a quello di produzione: ogni divergenza va documentata e motivata.

2.2.4 Strumenti a supporto

Utilizzare esclusivamente gli strumenti elencati. L'introduzione di uno strumento non presente in tabella richiede l'approvazione_G del Responsabile_G e l'aggiornamento di questa sezione.

Categoria	Strumento	Quando usarlo
Modellazione UML _G	StarUML	Diagrammi dei casi d'uso _G , delle classi, di sequenza e dei package
Ambiente di sviluppo _G	Visual Studio Code	Sviluppo _G di codice React e Python
Qualità del codice	ESLint, Prettier	Prima di ogni commit _G sul frontend _G
Qualità del codice	black	Prima di ogni commit _G sul backend
Testing frontend _G	Vitest, React Testing Library, Playwright	Test _G unitari, di componente e end-to-end
Testing backend	pytest	Test _G unitari e di integrazione _G
Containerizzazione	Docker, Docker Compose	Avvio dell'ambiente locale e di produzione
Integrazione continua _G	GitHub Actions	Automaticamente a ogni pull request _G
Versionamento _G	Git	Ogni modifica a codice o documentazione

Tabella 5: Strumenti a supporto dello sviluppo e criteri di impiego

Configurare la pipeline di integrazione continua_G in modo che a ogni pull request_G esegua, nell'ordine: lint, type checking, test_G unitari, verifica_G della copertura_G e build. Il fallimento di uno qualsiasi di questi passi blocca il merge_G.

3 Processi di supporto

I processi_G di supporto comprendono le attività trasversali necessarie a garantire il corretto svolgimento dei processi primari e organizzativi, assicurando che ogni artefatto_G prodotto sia conforme agli standard di qualità prefissati.

Tra i **processi**_G di supporto applicati nel progetto si distinguono:

3.1 Documentazione

3.1.1 Processo

Ruolo responsabile Redattore designato per il documento; Verificatore_G per la revisione; Responsabile_G per l'approvazione_G.

Input Issue_G che assegna la stesura o la modifica; template_G ufficiale; *Glossario* aggiornato.

Output Documento verificato_G, approvato_G, versionato_G e pubblicato nel branch_G principale.

Ogni documento prodotto è un artefatto_G ufficiale e deve rispettare, senza eccezioni, le cinque condizioni seguenti:

1. Raggiungere il livello minimo di qualità definito nel *Piano di Qualifica* per leggibilità, completezza e correttezza.
2. Adottare il template_G ufficiale, con la medesima struttura gerarchica e nomenclatura degli altri documenti.
3. Superare la verifica_G formale di un Verificatore_G diverso dal redattore, mediante la checklist di verifica_G documentale.
4. Registrare ogni modifica nel registro delle modifiche_G e incrementare la versione secondo le regole di versionamento_G.
5. Ottenere l'approvazione_G esplicita del Responsabile_G prima del consolidamento nel branch_G principale.

Regola ferrea: un documento privo anche di una sola delle cinque condizioni non è ufficiale e non può essere né consegnato né citato come riferimento.

Un processo documentale ben definito permette di: Mantenere ogni documento fedele allo stato corrente del progetto. In caso di discrepanza fra prodotto software e documentazione, correggere la documentazione entro lo sprint_G in corso: una discrepanza non sanata costituisce un difetto_G rilevabile in verifica_G.

3.1.1.1 Regole: Documentation as Code

Trattare la documentazione con gli stessi strumenti e le stesse procedure del codice sorgente. Applicare le regole seguenti senza eccezioni:

1. Conservare tutti i documenti nel repository_G di progetto, in cartelle separate per tipologia (verbali_G, documenti di progetto, documenti tecnici).
2. Redigere i documenti in L^AT_EX_G: è vietato produrre documenti ufficiali con altri formati.
3. Effettuare ogni modifica su un branch_G dedicato di tipo docs/, creato da develop.
4. Sottoporre ogni modifica a pull request_G, descrivendo contenuto e motivo dell'aggiornamento.
5. Far verificare_G ogni aggiornamento da un Verificatore_G prima del merge_G.
6. Delegare compilazione e pubblicazione del PDF alla pipeline CI/CD: è vietato caricare manualmente PDF generati in locale come versione ufficiale.
7. Aggiornare la documentazione nello **stesso sprint_G** in cui si modifica il prodotto che essa descrive.

3.1.2 Ciclo di vita dei documenti

Far attraversare a ogni documento le sette fasi seguenti, nell'ordine indicato. Non saltare fasi: il passaggio alla fase successiva richiede il completamento dell'output della precedente.

N.	Fase	Ruolo	Output che abilita la fase successiva
1	Necessità	Chiunque	Issue _G aperta che motiva la stesura o la modifica
2	Pianificazione _G	Responsabile _G	Redattore assegnato, struttura e scadenza definite
3	Redazione	Redattore	Contenuto completo secondo template _G e convenzioni
4	Verifica _G	Verificatore _G	Checklist di verifica _G documentale interamente soddisfatta
5	Approvazione _G	Responsabile _G	Riga di approvazione _G nel registro delle modifiche _G
6	Consolidamento	Amministratore _G	Merge _G in main e PDF generato dalla pipeline
7	Manutenzione	Redattore	Documento allineato allo stato corrente del progetto

Tabella 6: Ciclo di vita di un documento: fasi, ruoli e output

3.1.2.1 Fase 1 — Necessità

Aprire una issue_G documentale al verificarsi di una qualsiasi delle condizioni seguenti:

1. il capitolato_G o il calendario didattico impongono la consegna di un nuovo documento;
2. una decisione progettuale (tecnologia, pattern_G, protocollo) è stata assunta e va formalizzata;
3. un requisito_G è cambiato e impatta documenti di analisi o di progetto;
4. la proponente_G ha avanzato una richiesta durante un incontro o via email;
5. è stato rilevato un errore in un documento già approvato_G;
6. il prodotto è stato modificato e la documentazione non lo rispecchia più.

Output della fase: issue_G aperta che motiva la stesura o la modifica.

3.1.2.2 Fase 2 — Pianificazione

Ruolo responsabile Responsabile_G.

Input Issue_G documentale aperta.

Output Redattore e Verificatore_G assegnati; struttura e scadenza definite nella issue_G.

Passi da eseguire:

1. Stabilire se si tratta di un nuovo documento o della modifica di uno esistente.
2. Definire cosa il documento deve comunicare, a chi è rivolto e con quale livello di dettaglio.
3. Abbozzare l'indice individuando capitoli e paragrafi principali.

4. Fissare la scadenza di completamento, coerente con la pianificazione_G dello sprint_G.
5. Nominare il redattore e un Verificatore_G diverso da esso.
6. Assegnare la issue_G ai due incaricati e collegarla alla milestone_G di competenza.

3.1.2.3 Fase 3 — Redazione

Ruolo responsabile Redattore designato.

Input Issue_G assegnata; template_G ufficiale; *Glossario*.

Output Pull request_G aperta con il contenuto completo.

Passi da eseguire:

1. Creare un branch_G docs/ da develop.
2. Redigere il contenuto in L^AT_EX_G rispettando il template_G (frontespizio_G, registro delle modifiche_G, indice, corpo strutturato).
3. Marcare con il pedice _G ogni termine presente nel *Glossario*; aggiungere al *Glossario* i termini di dominio_G non ancora presenti.
4. Effettuare commit_G atomici, ciascuno riferito a un'unità logica di modifica.
5. Incrementare la versione e compilare il registro delle modifiche_G.
6. Compilare il documento in locale e verificare l'assenza di errori prima di aprire la pull request_G.

Requisiti di accettazione della redazione: uniformità terminologica rispetto al *Glossario*; coerenza di registro linguistico e formattazione; assenza di errori ortografici; conformità al template_G; assenza di sezioni incomplete o segnaposto.

3.1.2.4 Fase 4 — Verifica

Ruolo responsabile Verificatore_G designato, diverso dal redattore.

Input Pull request_G aperta dal redattore.

Output Approvazione_G della pull request_G oppure elenco di rilievi puntuali.

Applicare la checklist di verifica_G documentale definita nella Sezione 3.2, integrata dai controlli seguenti:

1. Verificare che il documento compili senza errori, in locale o tramite pipeline CI.
2. Controllare la resa finale del PDF: impaginazione, overflow di testo, leggibilità delle immagini.
3. Verificare che ogni rimando a sezioni, tabelle, figure e documenti esterni sia funzionante.
4. Segnalare ogni rilievo come commento inline riferito alla riga L^AT_EX_G interessata.

3.1.2.5 Fase 5 — Approvazione

Ruolo responsabile Responsabile_G.

Input Pull request_G approvata_G dal Verificatore_G.

Output Riga di approvazione_G nel registro delle modifiche_G; autorizzazione al merge_G.

Approvare_G un documento solo dopo aver accertato che la verifica_G si sia chiusa senza rilievi aperti. Con l'approvazione_G il documento diventa vincolante per le attività successive e utilizzabile come *ground truth*_G per sviluppo_G e verifica_G.

3.1.2.6 Fase 6 — Consolidamento

Ruolo responsabile Amministratore_G.

Input Pull request_G approvata_G dal Responsabile_G.

Output Documento in `main`; PDF definitivo generato dalla pipeline.

Eseguire il merge_G in `main`, attendere la generazione automatica del PDF e verificare che il documento pubblicato corrisponda alla versione approvata_G. Eliminare quindi il branch_G.

3.1.2.7 Fase 7 — Manutenzione

Riportare il documento alla **Fase 3 — Redazione** a ogni modifica, anche minima, e ripercorrere l'intero ciclo fino al consolidamento. Non è ammessa alcuna modifica diretta a un documento in `main`.

3.1.3 Attività previste

3.1.3.1 Produzione della documentazione

La produzione comprende tutte le attività di:

- **stesura**: scrittura di nuovi contenuti, seguendo i template e le linee guida;
- **aggiornamento**: modifica di documenti esistenti per adeguarli all'evoluzione del progetto;
- **manutenzione**: interventi correttivi e migliorativi successivi all'approvazione_G;
- **consolidamento**: integrazione_G delle versioni approvate nel ramo principale e generazione dei PDF ufficiali.

Il gruppo utilizza $\text{\LaTeX}$ _G come linguaggio di markup ufficiale per:

- garantire elevata qualità tipografica, professionale e adatta a documenti tecnici;
- separare contenuto e presentazione, permettendo di concentrarsi sulla scrittura senza preoccuparsi dell'impaginazione;
- facilitare il versionamento tramite Git_G, poiché i file `.tex` sono testo puro e quindi facilmente confrontabili;
- automatizzare la generazione di indici, riferimenti incrociati, bibliografia e glossario, riducendo il lavoro manuale e il rischio_G di errori;
- uniformare la struttura dei documenti, grazie all'uso di classi e comandi personalizzati condivisi tra tutti i documenti del gruppo.

3.1.3.2 Revisione

Ogni modifica deve essere sottoposta a revisione formale prima dell'integrazione_G nei branch_G principali. La revisione non è un controllo superficiale, ma un'attività strutturata che segue una checklist definita.

La revisione ha lo scopo di:

- prevenire incoerenze tra le diverse parti del documento e tra documenti diversi;
- garantire conformità alle NdP_G, verificando il rispetto di template, nomenclatura e convenzioni;
- identificare errori di contenuto, come dati errati, calcoli sbagliati o affermazioni non supportate;
- validare i contenuti prodotti, assicurandosi che riflettano fedelmente lo stato del progetto e le decisioni prese.

3.1.3.3 Approvazione

L'approvazione_G finale dei documenti è responsabilità del Responsabile_G. Questi non può delegare tale compito, poiché l'approvazione_G implica l'assunzione di responsabilità verso il proponente_G e verso il gruppo.

Il Responsabile_G:

- valuta il risultato della verifica_G, esaminando i commenti del verificatore_G e accertandosi che tutte le osservazioni siano state risolte;
- approva formalmente il documento, tramite l'accettazione della Pull Request_G di approvazione_G;
- autorizza il merge_G finale, garantendo che il documento approvato venga effettivamente consolidato;
- richiede eventuali ulteriori modifiche, se ritiene che il documento non abbia ancora raggiunto il livello qualitativo atteso, respingendo la Pull Request_G e motivando la decisione.

3.1.3.4 Archiviazione e pubblicazione

I documenti approvati vengono:

- archiviati nel branch_G `main`, che contiene esclusivamente versioni stabili e verificate;
- pubblicati tramite GitHub_G Pages_G, che rende disponibile un sito web con l'elenco dei documenti e i link ai PDF scaricabili;
- conservati come riferimento ufficiale del progetto, in modo che ogni stakeholder_G possa sempre accedere alla versione più aggiornata.

3.1.3.5 Matrice delle responsabilità documentali

Attribuire i compiti del processo documentale secondo la matrice seguente. Nessun compito può essere svolto da un ruolo_G diverso da quello indicato.

Compito	Ruolo	Quando
Individuare i documenti da produrre	Responsabile _G	Apertura sprint _G
Stabilire priorità e scadenze	Responsabile _G	Apertura sprint _G
Assegnare redattore e Verificatore _G	Responsabile _G	Apertura sprint _G
Supervisionare l'avanzamento e intervenire sui ritardi	Responsabile _G	Continuativo
Approvare _G formalmente il documento	Responsabile _G	A verifica _G conclusa
Redigere i verbali _G interni ed esterni	Responsabile _G	Entro 48 ore da ogni riunione
Creare le issue _G documentali nell'ITS _G	Amministratore _G	Entro 24 ore dalla riunione
Gestire struttura, permessi e protezione dei branch _G	Amministratore _G	Continuativo
Monitorare versioni e registri delle modifiche _G	Amministratore _G	Settimanale
Redigere l' <i>Analisi dei Requisiti</i>	Analista _G	Fase di analisi
Verificare _G i documenti	Verificatore _G	A ogni pull request _G

Tabella 7: Matrice delle responsabilità nel processo documentale

Regola ferrea: l'approvazione_G dei documenti non è delegabile. Solo il Responsabile_G in carica può approvare_G, poiché con tale atto assume responsabilità verso la proponente_G e verso il gruppo.

3.1.3.6 Redazione dei verbali

Redigere i verbali_G a partire dalle discussioni dei meeting interni, dagli incontri con la proponente_G e dalle decisioni assunte durante le attività operative. Applicare la struttura definita nella Sezione 3.1.4.2 e la procedura di verbalizzazione definita nella Sezione 2.1.4.3.

3.1.3.7 Redazione dell'Analisi dei Requisiti

Affidare la redazione esclusivamente all'Analista_G, che deve:

1. **Identificare** i requisiti_G analizzando il capitolato_G, i verbali_G degli incontri con la proponente_G e sistemi_G analoghi.
2. **Classificare** ciascun requisito_G come funzionale, qualitativo o di vincolo_G, assegnandogli un codice univoco.
3. **Formalizzare** il requisito_G in linguaggio naturale controllato, secondo il template_G predefinito.
4. **Controllare** che ogni requisito_G sia coerente, non ambiguo, verificabile_G e completo prima di sottoporlo ad approvazione_G.

3.1.4 Procedure operative

3.1.4.1 Struttura generale dei documenti

Tutti i documenti ufficiali devono rispettare una struttura comune, che garantisce uniformità e facilita la consultazione.

3.1.4.1.1 Frontespizio La prima pagina del documento deve contenere le seguenti informazioni, disposte secondo il template $\text{\LaTeX}_G$ condiviso:

- logo ufficiale del gruppo, posizionato al centro;
- titolo del documento, in carattere grande e grassetto;
- nome del gruppo, come da registrazione ufficiale;
- identificativo del gruppo;
- elenco dei componenti ordinati in ordine alfabetico, con nome e cognome;
- numero di matricola di ciascun componente, per identificazione univoca;
- versione del documento, nel formato X.Y.Z secondo il semantic versioning $_G$;
- data di rilascio, nel formato AAAA-MM-GG;
- stato del documento (Stesura, Revisione, Approvato);
- destinatari, ovvero i soggetti a cui il documento è indirizzato;
- indirizzo e-mail ufficiale del gruppo, per eventuali comunicazioni.

3.1.4.1.2 Registro delle modifiche Il registro delle modifiche:

- viene posizionato a partire dalla seconda pagina, immediatamente dopo il frontespizio $_G$;
- segue logica LIFO $_G$: la modifica più recente compare in cima alla tabella, facilitando la lettura delle ultime variazioni;
- mantiene tracciabilità $_G$ delle revisioni effettuate, costituendo un riepilogo cronologico dell'evoluzione del documento.

Per ogni modifica devono essere riportati obbligatoriamente:

- **versione**: il numero di versione dopo la modifica;
- **data**: giorno in cui la modifica è stata completata;
- **autore**: nome del redattore che ha apportato la modifica;
- **verificatore $_G$** : nome di chi ha effettuato la revisione;
- **descrizione delle modifiche**: sintesi chiara ed esaustiva di cosa è stato cambiato, con eventuale riferimento alla Issue $_G$ associata.

3.1.4.1.3 Indice Ogni documento deve contenere un indice gerarchico $_G$ generato automaticamente da $\text{\LaTeX}_G$, dotato di collegamenti ipertestuali $_G$ alle relative sezioni. L'indice consente una navigazione rapida e viene posto dopo il registro delle modifiche.

3.1.4.2 Struttura dei Verbali

I verbali costituiscono un'eccezione rispetto alla struttura generale definita in 3.1.4.1. Essi non includono il registro delle modifiche. Per loro natura di documento puntuale che fotografa una specifica riunione, l'ultima versione approvata sostituisce integralmente le precedenti; eventuali correzioni vengono incorporate direttamente in una nuova versione del verbale, senza necessità di conservare una cronologia delle revisioni intermedie.

In aggiunta al template standard, i verbali devono presentare una struttura standard, pensata per rendere rapida la consultazione delle informazioni salienti. La struttura è composta dalle seguenti sezioni obbligatorie.

3.1.4.2.1 Informazioni sulla riunione

Devono essere specificati:

- **data:** giorno in cui si è svolta la riunione;
- **orario di inizio:** ora esatta di avvio;
- **orario di conclusione:** ora di termine, per quantificare la durata effettiva;
- **modalità di svolgimento:** indicazione se in presenza, tramite videochiamata (specificando la piattaforma).

3.1.4.2.2 Presenze

La sezione deve riportare:

- nome e cognome dei partecipanti, ordinati alfabeticamente per cognome;
- stato di presenza (Presente/Assente);
- eventuali soggetti esterni e relativa azienda, se hanno preso parte alla riunione.

3.1.4.2.3 Ordine del giorno Deve contenere l'elenco puntato degli argomenti previsti, nella forma in cui è stato comunicato ai partecipanti prima della riunione. Durante la riunione possono essere affrontati anche argomenti non previsti, che vanno aggiunti in un paragrafo separato "Varie ed eventuali".

3.1.4.2.4 Discussione e sintesi

Contiene:

- resoconto delle discussioni, con una sintesi per ogni punto all'ordine del giorno;
- decisioni prese, evidenziate con un formato che le renda immediatamente individuabili (es. grassetto o rientro);
- brainstorming_G effettuato, con la registrazione delle idee emerse e delle valutazioni preliminari;
- problematiche emerse, con l'indicazione di eventuali soluzioni proposte o della necessità di approfondimenti.

3.1.4.2.5 Attività da svolgere La sezione deve includere, per ogni attività individuata:

- task_G assegnati, con una descrizione sintetica ma non ambigua;
- responsabili, ovvero i membri incaricati di portare a termine il compito;
- scadenze, espresse come data entro cui il task_G deve essere completato;
- riferimenti alle Issue_G create sull' ITS_G , per consentire il tracciamento automatico dello stato di avanzamento.

3.1.4.3 Versionamento

Per il versionamento della documentazione il gruppo utilizza Git_G , sfruttando appieno le sue capacità di branching e merging.

3.1.4.3.1 Branch Ogni modifica documentale deve essere effettuata su branch_G dedicati derivati da `develop`. Si opera sempre su un branch_G separato per isolare le modifiche fino al completamento della revisione.

Il workflow $_G$ adottato prevede i seguenti passi sequenziali:

1. **creazione del branch:** il redattore crea un branch_G con nome descrittivo (es. `docs/pdp-pianificazione`) a partire da `develop`;
2. **modifica del documento:** il redattore apporta le modifiche necessarie, facendo commit frequenti e ben commentati;
3. **commit $_G$ delle modifiche:** ogni commit è atomico e descrive in modo chiaro la modifica effettuata;
4. **push sul repository remoto:** il branch_G viene caricato su GitHub_G per consentire la condivisione e l'apertura della Pull Request $_G$;
5. **apertura della Pull Request $_G$:** il redattore crea una PR_G verso `develop`, descrivendo le modifiche e collegando la Issue_G di riferimento;
6. **assegnazione del Verificatore $_G$:** la PR_G viene assegnata al verificatore $_G$ designato, che riceve notifica e può procedere con la revisione.

3.1.4.3.2 Stato dei documenti Ogni documento può assumere uno dei seguenti stati, che devono essere indicati nel frontespizio $_G$ e sulla dashboard $_G$ di progetto:

- **Stesura:** il documento è in fase di redazione e non è ancora stato sottoposto a verifica $_G$ formale;
- **Revisione:** il documento è stato inviato in Pull Request $_G$ ed è in attesa del responso del verificatore $_G$;
- **Approvato:** il documento ha superato verifica $_G$ e approvazione $_G$ ed è stato consolidato nel branch_G `main`.

Solo i documenti in stato "Approvato" sono considerati ground truth $_G$ e possono essere utilizzati come riferimento vincolante per le attività di progetto.

3.1.4.3.3 Nomenclatura Le convenzioni di nomenclatura adottate garantiscono l'immediata riconoscibilità del tipo di documento e della sua versione. Esse sono:

- **Verbali Interni:** VI_AAAA-MM-GG_descrizione-breve.pdf
dove AAAA-MM-GG è la data della riunione e la descrizione breve riassume l'oggetto principale.
- **Verbali Esterni:** VE_AAAA-MM-GG_descrizione-breve.pdf
analogo al precedente, ma con prefisso VE.
- **Documenti Generali:** nome-documento_vX.Y.Z.pdf
dove nome-documento è un identificativo univoco (es. Norme-di-Progetto) e X.Y.Z è il numero di versione.

3.1.4.3.4 Commit e Pull Request Ogni commit_G e Pull Request_G deve rispettare regole precise per garantire tracciabilità e comprensibilità:

- introdurre modifiche significative: non sono ammessi commit vuoti o con modifiche irrilevanti; ogni commit deve rappresentare un progresso tangibile;
- essere associato a una Issue_G : nel messaggio di commit e nella descrizione della PR_G deve comparire il riferimento alla Issue_G (es. `Closes #42`) per collegare automaticamente la modifica al task_G ;
- possedere descrizioni chiare e comprensibili: il messaggio di commit deve riassumere in modo conciso la modifica, mentre la descrizione della PR_G può fornire dettagli aggiuntivi, motivazioni e screenshot;
- rispettare le convenzioni stabilite dal gruppo: in particolare, seguire il formato `<tipo>: <descrizione>` per i commit (es. docs: aggiorna procedura di verifica).

3.1.4.4 Processo di verifica tramite Pull Request

Il workflow_G di verifica_G documentale, già descritto nella sezione dedicata alla verifica_G, si applica integralmente ai documenti e si articola nei seguenti passi:

1. **creazione del branch dedicato:** il redattore crea un branch_G a partire da `develop`;
2. **modifica del documento:** vengono apportate le modifiche richieste;
3. **push delle modifiche:** il branch_G viene caricato sul repository_G remoto;
4. **apertura della Pull Request_G:** su GitHub_G, il redattore crea una PR_G verso `develop`, compilando il template di descrizione;
5. **assegnazione del Verificatore_G:** la PR_G viene assegnata manualmente o automaticamente al verificatore_G designato;
6. **collegamento dell'Issue_G pertinente:** nella PR_G si inserisce il riferimento alla Issue_G da chiudere al momento del merge_G.

Il verificatore_G, durante la revisione:

- controlla il contenuto del documento, riga per riga, basandosi sulla checklist di verifica_G;

- inserisce commenti inline direttamente sul codice $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}_G$ modificato, in modo da rendere immediato il contesto della segnalazione;
- approva la PR_G se tutte le modifiche sono conformi, oppure richiede modifiche, indicando esattamente quali punti devono essere corretti.

In caso di richiesta di modifiche, il redattore le apporta, aggiorna il branch_G e la PR_G viene riesaminata fino all'approvazione G finale.

3.1.4.5 Processo di approvazione tramite Pull Request

L'approvazione G costituisce l'atto formale che consente il rilascio ufficiale della documentazione. Si distingue dalla verifica G perché coinvolge il Responsabile G , che valuta il documento nella sua interezza e ne autorizza la pubblicazione.

La procedura adottata per l'approvazione G è:

1. **creazione del branch:** si crea un branch_G dedicato denominato `chore/approvazione-<nome-documento>` a partire da `develop`;
2. **aggiornamento della versione Major:** all'interno del documento si incrementa la prima cifra della versione (es. da 1.2.3 a 2.0.0) per segnalare un cambiamento sostanziale, e si aggiorna il registro delle modifiche;
3. **apertura della Pull Request G :** si apre una PR_G verso `develop` descrivendo che si tratta della richiesta di approvazione G ;
4. **assegnazione del Responsabile G :** la PR_G viene assegnata al Responsabile G , unico autorizzato a effettuare il merge_G di approvazione G ;
5. **approvazione finale:** il Responsabile G esamina il documento (eventualmente consultando il verificatore G) e, se lo ritiene adeguato, approva la PR_G ;
6. **merge G nel branch `develop`:** la PR_G viene mergiata, portando il documento approvato nel ramo di sviluppo G ;
7. **consolidamento nel branch `main`:** successivamente, si effettua un merge_G da `develop` a `main`, rendendo la nuova versione disponibile come versione ufficiale e attivando la generazione del PDF definitivo.

Questo flusso garantisce che solo documenti completamente verificati e approvati confluiscono nel ramo principale.

3.1.4.6 Acronimi

La tabella 8 riporta gli acronimi utilizzati in modo ricorrente nella documentazione, con il relativo significato esteso. Essi devono essere espansi alla prima occorrenza in ogni documento.

Acronimo	Significato
VI	Verbale Interno
VE	Verbale Esterno
NdP	Norme di Progetto _G
AdR	Analisi dei Requisiti _G
PdP	Piano di Progetto _G
PdQ	Piano di Qualifica _G
RTB	Requirements and Technology Baseline _G
PB	Product Baseline _G

Tabella 8: Acronimi utilizzati nella documentazione.

3.1.5 Strumenti di supporto

3.1.5.1 L^AT_EX_G

L^AT_EX_G è il linguaggio di markup adottato per la produzione della documentazione ufficiale. La scelta è motivata dalla sua capacità di produrre documenti di elevata qualità tipografica, paragonabile a quella delle pubblicazioni scientifiche.

L'utilizzo di L^AT_EX_G consente:

- **elevata qualità tipografica:** gestione automatica della sillabazione, della giustificazione e della spaziatura, con un risultato professionale e uniforme;
- **gestione automatica di riferimenti e indice:** etichette e riferimenti incrociati (label/ref) eliminano il rischio_G di rimandi errati quando si inseriscono o spostano sezioni;
- **integrazione con Git_G:** i file `.tex` sono testo semplice, quindi versionabili, confrontabili e mergeabili con gli stessi strumenti del codice;
- **facilità di manutenzione:** la separazione tra struttura e contenuto, insieme all'uso di comandi personalizzati, permette di modificare lo stile dell'intera documentazione agendo su pochi file di configurazione.

3.1.5.2 GitHub_G

GitHub_G è la piattaforma di hosting scelta per tutti i repository_G del progetto. Oltre al versionamento del codice, essa supporta integralmente il processo documentale grazie alle seguenti funzionalità:

- **versionamento:** ogni modifica ai documenti è tracciata tramite commit_G, con possibilità di revert e confronto tra versioni;
- **gestione delle Pull Request_G:** la revisione documentale si svolge interamente su PR_G, con commenti, approvazioni e merge_G;
- **gestione delle Issue_G:** le attività documentali sono tracciate come Issue_G, assegnate ai membri e collegate a milestone_G;
- **hosting della documentazione:** i PDF generati sono pubblicati tramite GitHub_G Pages_G, rendendoli accessibili a tutti gli stakeholder_G;

- **tracciamento delle modifiche:** la timeline delle attività e la history dei file consentono di ricostruire l'evoluzione di ogni documento.

Le funzionalità maggiormente utilizzate nel processo documentale sono:

- **GitHub Issues:** per la gestione dei task_G di redazione, modifica e verifica_G;
- **GitHub Projects:** per la visualizzazione Kanban dello stato dei documenti (Stesura, Revisione, Approvato);
- **GitHub Pages:** per la pubblicazione automatica dei PDF ufficiali e per l'hosting del sito di documentazione del progetto.

3.1.5.3 Visual Studio Code

Visual Studio Code è l'editor di testo scelto dal gruppo per la redazione dei documenti in $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}_G$. Grazie al suo ecosistema di estensioni e all'integrazione_G nativa con Git_G, supporta efficacemente il workflow_G documentale adottato.

3.1.5.4 Notion

Notion è un ambiente collaborativo flessibile, impiegato come spazio di lavoro informale per:

- **brainstorming_G:** raccolta di idee, appunti non strutturati e prime bozze di contenuti;
- **stesura preliminare:** scrittura di prime versioni di testi, tabelle e diagrammi in un contesto collaborativo a bassa formalità, da trasferire successivamente in $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}_G$;
- **checklist documentali:** elenchi di controllo per la redazione e la verifica_G, consultabili e spuntabili da tutti i membri.

L'uso di Notion è limitato alla fase preparatoria e di raccolta; tutti i contenuti destinati a diventare ufficiali devono essere convertiti in $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}_G$ e inseriti nella repository_G di progetto.

3.2 Verifica

3.2.1 Processo di verifica

Ruolo responsabile Verificatore_G.

Input Artefatto_G dichiarato completo dal suo autore; criteri di accettazione definiti nel *Piano di Qualifica*.

Output Esito di verifica_G registrato nel *Piano di Qualifica*; elenco dei rilievi con relativa gravità; artefatto_G approvato_G oppure respinto.

Attivare la verifica_G di un artefatto_G in una sola delle due condizioni seguenti:

1. l'autore dichiara l'artefatto_G completo e apre la relativa pull request_G;
2. l'artefatto_G subisce una modifica dopo essere entrato in baseline_G.

Regole ferree del processo di verifica_G:

1. Verificare sempre rispetto a **criteri oggettivi** predefiniti: è vietato respingere un artefatto_G sulla base di preferenze personali non riconducibili a una norma di questo documento.
2. Non verificare artefatti_G di cui si è autori: il Verificatore_G deve essere persona diversa dal redattore.
3. Registrare nel *Piano di Qualifica* obiettivi, risultati attesi e risultati ottenuti di ogni attività di verifica_G.
4. Motivare ogni rilievo indicando la norma violata e l'azione correttiva richiesta.
5. Ripetere la verifica_G dopo ogni correzione, fino ad assenza di rilievi.

3.2.2 Attività di verifica

3.2.2.1 Procedura: verifica di un documento

Ruolo responsabile Verificatore_G, diverso dal redattore.

Input Pull request_G aperta sul documento; issue_G nella colonna “Da revisionare”.

Output Documento approvato_G, oppure elenco di rilievi puntuali sulla pull request_G.

Passi da eseguire:

1. Aprire la pull request_G del branch_G di redazione verso il ramo di destinazione.
2. Applicare la checklist di verifica_G documentale riportata di seguito, voce per voce.
3. Annotare ogni rilievo come commento puntuale sulla riga interessata, indicando la norma violata.
4. Richiedere modifiche se anche una sola voce risulta non soddisfatta; approvare_G solo in caso contrario.
5. Registrare l'esito nel *Piano di Qualifica*.
6. Ripetere la checklist dopo ogni correzione dell'autore.

Checklist di verifica documentale:

N.	Voce da verificare
1	Le informazioni sono tecnicamente esatte e coerenti con gli altri documenti
2	La struttura rispetta il template _G e le convenzioni di formattazione
3	Non sussistono contraddizioni interne né con documenti correlati
4	Il linguaggio è privo di ambiguità e adeguato al destinatario
5	Il testo è corretto sul piano grammaticale e ortografico
6	I termini di dominio _G sono marcati con il pedice _G e presenti nel <i>Glossario</i>
7	Il registro delle modifiche _G è compilato correttamente, con il campo <i>Approvatore</i> valorizzato solo nelle righe di approvazione _G
8	La versione è stata incrementata secondo le regole di versionamento _G
9	Tabelle e diagrammi sono numerati, referenziati nel testo e leggibili
10	Il documento compila senza errori

Tabella 9: Checklist di verifica di un documento

3.2.2.2 Procedura: analisi statica

Condurre l'analisi statica sull'artefatto_G senza eseguirlo, per individuare errori formali, violazioni di vincoli_G e difetti_G. Selezionare la tecnica secondo il criterio seguente:

Tecnica	Applicare quando	Applicare a
Walkthrough	L'artefatto _G è semplice e la ricerca dei difetti _G non è mirata a una tipologia specifica	Requisiti _G , documentazione tecnica, codice nelle fasi iniziali
Inspection	L'artefatto _G è complesso e strutturato, e sono note le tipologie di difetto _G da cercare	Codice e documenti nelle fasi avanzate

Tabella 10: Criterio di selezione della tecnica di analisi statica

Walkthrough — eseguire i quattro passi nell'ordine:

1. **Pianificare:** concordare con l'autore le proprietà e i vincoli_G di correttezza da controllare.
2. **Leggere:** analizzare l'artefatto_G rilevando gli scostamenti dai vincoli_G stabiliti.
3. **Discutere:** confrontarsi con l'autore sugli esiti e concordare le correzioni.
4. **Correggere:** l'autore sana i difetti_G e restituisce l'artefatto_G alla verifica_G.

Inspection — eseguire i passi seguenti:

1. Definire prima dell'ispezione la checklist degli elementi da controllare.
2. Esaminare l'artefatto_G voce per voce, senza deviare dalla checklist.
3. Registrare ogni difetto_G rilevato con riferimento alla voce di checklist violata.

3.2.2.3 Procedura: analisi dinamica e testing

Applicare l'analisi dinamica esclusivamente al software, mai alla documentazione.

Automatizzare ogni test_G: un test_G non ripetibile non è considerato valido.

Eseguire i livelli di test_G nell'ordine seguente. Non procedere al livello successivo prima del superamento del precedente.

Ord.	Livello	Verificare	Pianificare in	Criterio di superamento
1	Unità	Singole funzioni o metodi in isolamento, con mock e stub	Progettazione _G di dettaglio	Copertura _G dei rami $\geq 80\%$ e tutti i test _G verdi
2	Integrazione _G	Interazione fra unità aggregate, con approccio top-down	Progettazione _G architetturale	Interfacce fra moduli _G corrette e tutti i test _G verdi
3	Regressione	Assenza di difetti _G introdotti da modifiche	Continuativamente	Nessuna funzionalità preesistente compromessa
4	Sistema _G	Sistema _G integrato rispetto ai requisiti _G , con flussi <i>end-to-end</i>	Analisi dei requisiti _G	Tutti i requisiti _G obbligatori soddisfatti
5	Accettazione	Conformità alle attese della proponente _G	Contratto	Accettazione formale verbalizzata

Tabella 11: Livelli di test: ordine di esecuzione e criteri di superamento

Regole ferree sul testing:

1. Eseguire i test_G di regressione a **ogni** modifica del codice, in modo automatico tramite la pipeline di integrazione continua_G.
2. Scrivere test_G indipendenti fra loro, ripetibili e privi di dipendenze da stato globale o da risorse esterne non mockate.
3. Non disattivare né ignorare un test_G fallito: correggere il codice oppure, se il test_G è errato, correggere il test_G documentandone la ragione nella pull request_G.

3.2.2.4 Test di accettazione

- Sono l'ultimo passo prima del rilascio.

- Verificano che il prodotto finale soddisfi le aspettative degli utenti e risponda adeguatamente ai requisiti_G specificati, tipicamente attraverso l'esecuzione di scenari d'uso concordati con il proponente_G.

3.2.2.5 Sequenza delle fasi di test

La sequenza operativa delle fasi di test_G è la seguente:

1. Test_G di unità;
2. Test_G di integrazione_G;
3. Test_G di regressione (da ripetere a ogni modifica);
4. Test_G di sistema_G;
5. Test_G di accettazione.

3.2.2.6 Codici identificativi dei test

Ogni test_G è identificato da un codice univoco nel formato T[tipo][codice], dove:

- [tipo] può essere:
 - U – test_G di unità;
 - I – test_G di integrazione_G;
 - S – test_G di sistema_G;
 - R – test_G di regressione;
 - A – test_G di accettazione.
- [codice] è un numero progressivo.
 - Se il test_G è figlio di un altro test_G, il codice segue il formato [codice_padre].[codice_figlio],
 - dove codice_padre identifica univocamente il test_G genitore all'interno della stessa categoria.

3.2.2.7 Stato dei test

A ogni test_G viene assegnato uno stato che ne riflette l'esito, registrato nel *Piano di Qualifica* (sezione "Specifiche dei test_G"):

- **NI** (Non Implementato): il test_G non è ancora stato realizzato.
- **S** (Superato): esito positivo.
- **NS** (Non Superato): esito negativo.

3.2.2.8 Integrazione continua

L'integrazione_G continua (*Continuous Integration* – CI) è una pratica di sviluppo_G che automatizza l'integrazione_G frequente del codice sorgente.

- Invece di attendere il completamento di grandi porzioni di lavoro, gli sviluppatori integrano le modifiche nel ramo principale più volte al giorno.
- Nel progetto, la CI è attivata dalla creazione di una *Pull Request*: quando un programmatore_G vuole unire le modifiche di un branch_G dedicato al ramo principale, una pipeline automatica (GitHub_G Actions) esegue i test_G definiti, fornendo un feedback_G immediato sulla qualità dell'integrazione_G.

3.2.3 Procedure operative

3.2.3.1 Procedura di verifica documentale

1. Il verificatore_G riceve notifica della *Pull Request* e accede alla pagina della PR_G nel repository_G **Sorgente-documenti**.
2. Esamina il documento, aggiunge commenti e, se necessario, richiede modifiche.
3. Una volta constatato il rispetto dei criteri di qualità, il verificatore_G:
 - accetta la *Pull Request*, risolve eventuali conflitti ed esegue il *merge* nel branch_G **main**;
 - elimina il branch_G di redazione;
 - sposta la *issue* corrispondente dalla colonna “Da revisionare” a “Done” nella Dashboard_G documentazione.
4. Un'automazione GitHub_G Actions compila il sorgente L^AT_EX_G e genera il PDF nel branch_G **main** del repository_G **Documents**.
 - Per i verbali il PDF riporterà la data di redazione;
 - per tutti gli altri documenti la versione aggiornata.
5. Se la compilazione fallisce, il verificatore_G riceve una notifica via email e deve consultare la sezione “Actions” di GitHub_G per diagnosticare e correggere l'errore.

3.2.3.2 Procedura di walkthrough

1. **Pianificazione**: incontro tra autore e verificatore_G per definire le proprietà e i vincoli di correttezza.
2. **Lettura**: il verificatore_G analizza il prodotto nella sua interezza, segnando gli errori e le non conformità.
3. **Discussione**: incontro di confronto per analizzare gli esiti della lettura e concordare le correzioni.
4. **Correzione**: l'autore applica le modifiche richieste.

La procedura viene eseguita per ogni prodotto soggetto a walkthrough, tipicamente nelle fasi iniziali del progetto.

3.2.4 Strumenti di supporto

Per automatizzare e rendere ripetibili le attività di verifica_G dinamica, il team si avvale di strumenti specifici per il frontend_G e per il backend.

3.2.4.1 Test frontend

- **Vitest**
 - Test_G runner veloce, nativamente integrato con l'ecosistema Vite.
 - Supporta TypeScript senza configurazione aggiuntiva e offre funzionalità moderne come l'*Hot Module Replacement* durante l'esecuzione dei test_G.
 - Viene impiegato per eseguire i test_G di unità e di integrazione_G del frontend_G, sfruttando la sua rapidità e la piena compatibilità con il bundler di progetto.
- **React Testing Library**
 - Fornisce utility per testare componenti React concentrandosi sul comportamento visibile all'utente, anziché sui dettagli implementativi.
 - I test_G interrogano il DOM nello stesso modo in cui lo farebbe un utente (ad esempio cercando elementi per etichetta, ruolo o testo).
 - Viene utilizzato in combinazione con Vitest per scrivere test_G di unità e integrazione_G che simulano l'interazione reale con l'interfaccia.
- **Playwright**
 - Framework_G di test_G end-to-end che automatizza i browser (Chromium, Firefox, WebKit).
 - Consente di simulare flussi utente completi, interazioni complesse e di verificare l'applicazione in ambienti multi-browser.
 - Viene adottato per i test_G di sistema_G e di accettazione del frontend_G, perché offre un'elevata fedeltà rispetto all'esperienza reale dell'utente finale e si integra con pipeline CI senza richiedere display grafico.

3.2.4.2 Test backend

- **pytest**
 - Potente framework_G di test_G per Python, caratterizzato da una sintassi concisa e dalla ricchezza di plugin.
 - Supporta *fixtures* per la gestione dello stato, parametrizzazione dei test_G e asserzioni estese.
 - Viene scelto per tutti i livelli di test_G del backend (unità, integrazione_G e sistema_G) perché permette di scrivere test_G chiari, facilmente manutenibili e di integrarli senza sforzo nella pipeline di integrazione_G continua.
 - La possibilità di marcare i test_G (es. `@pytest.mark.unit`, `@pytest.mark.integration`) consente di eseguire selettivamente le suite in base al contesto.

3.3 Validazione

3.3.1 Processo di validazione

Ruolo responsabile Verificatore_G per l'esecuzione dei test_G; Responsabile_G per la dimostrazione alla proponente_G.

Input Prodotto che ha superato la verifica_G; *Analisi dei Requisiti* approvata_G; ambiente di destinazione predisposto.

Output Verbale_G esterno che attesta l'esito della validazione_G; esiti dei test_G di accettazione registrati nel *Piano di Qualifica*.

Condizioni per dichiarare validato il prodotto. Dichiarare il prodotto validato_G solo se **entrambe** le condizioni seguenti risultano soddisfatte:

1. tutti i requisiti_G obbligatori e desiderabili concordati con la proponente_G sono soddisfatti e documentati nell'*Analisi dei Requisiti*;
2. il prodotto è eseguibile correttamente in una configurazione_G che riproduce fedelmente l'ambiente di utilizzo reale.

In assenza anche di una sola condizione, registrare le non conformità come issue_G e ripetere il ciclo di sviluppo_G, verifica_G e validazione_G.

3.3.2 Attività di validazione

Eseguire le cinque attività seguenti, nell'ordine indicato.

1. **Eseguire i test_G di accettazione** — *Ruolo*: Verificatore_G. Eseguire tutti i test_G di tipo A nell'ambiente di produzione simulato, verificando la risposta del sistema_G agli scenari d'uso concordati con la proponente_G. *Output*: esiti S (Superato) / NS (Non Superato) registrati.
2. **Valutare la copertura_G dei casi d'uso_G** — *Ruolo*: Verificatore_G. Accertare che ogni caso d'uso_G dell'*Analisi dei Requisiti* sia implementato e superi i relativi test_G. *Output*: tabella di copertura_G caso d'uso-test_G completa.
3. **Verificare i requisiti_G obbligatori** — *Ruolo*: Verificatore_G. Accertare che il 100% dei requisiti_G obbligatori sia realizzato e superi i test_G. Valutare separatamente desiderabili e opzionali per quantificare il livello di completezza raggiunto. *Output*: percentuale di soddisfacimento per ciascuna classe di priorità.
4. **Dimostrare il prodotto alla proponente_G** — *Ruolo*: Responsabile_G. Organizzare la sessione, mostrare il funzionamento del sistema_G e illustrare gli esiti dei test_G. *Output*: verbale_G esterno della dimostrazione.
5. **Gestire i riscontri** — *Ruolo*: Responsabile_G. Registrare come issue_G ogni osservazione, richiesta di modifica o nuovo requisito_G emerso; sottoporli alla procedura di Change Request. *Output*: issue_G aperte e classificate per priorità.

3.3.3 Procedure operative

Il processo di validazione_G è formalizzato attraverso una procedura operativa che ne garantisce la ripetibilità e la tracciabilità_G.

1. **Preparazione dell'ambiente:** l'ambiente di test_G viene configurato in modo da riprodurre il più fedelmente possibile l'ambiente di utilizzo finale (stesso sistema_G operativo, stesse dipendenze software_G, stessa configurazione di rete). I dati utilizzati sono dati di test_G rappresentativi, privi di informazioni sensibili.
2. **Esecuzione della suite di accettazione:** i test_G di accettazione (codificati come TA) vengono eseguiti. L'esecuzione può essere automatizzata tramite gli strumenti di testing. I risultati sono registrati automaticamente e riportano lo stato **S** (Superato) o **NS** (Non Superato).
3. **Registrazione degli esiti:** i risultati dei test_G di accettazione sono documentati nel *Piano di Qualifica*, nella sezione dedicata alla specifica dei test_G, aggiornando lo stato di ciascun test_G.
4. **Sessione di dimostrazione:** il Responsabile_G convoca una riunione con il proponente_G, durante la quale:
 - vengono mostrate le funzionalità implementate;
 - vengono illustrati i risultati dei test_G di accettazione;
 - si discutono eventuali criticità o aspetti da migliorare.
5. **Approvazione formale:** se il proponente_G ritiene il prodotto conforme alle aspettative, viene redatto un verbale esterno che attesta l'esito positivo della validazione_G. Tale verbale costituisce l'evidenza oggettiva del completamento del processo.
6. **Gestione delle non conformità:** qualora emergano difetti, funzionalità mancanti o nuovi requisiti_G, essi vengono registrati nell'ITS_G come Issue_G, valutati dal gruppo e, se accettati, portati in un nuovo ciclo di sviluppo_G, verifica_G e successiva validazione_G. La procedura riprende quindi dal punto 1 per la nuova versione del prodotto.
7. **Rilascio:** una volta ottenuta l'approvazione_G formale e completata la validazione_G, il prodotto viene considerato pronto per il rilascio. Il branch_G **main** viene aggiornato con la versione finale e il team procede con le eventuali attività di deployment concordate.

3.3.4 Strumenti di supporto

La validazione_G si avvale degli stessi strumenti di testing utilizzati per il processo di verifica_G (§3.2.5), integrati con strumenti di comunicazione e tracciamento.

- **Playwright:** impiegato per l'esecuzione automatizzata dei test_G end-to-end e di accettazione del frontend_G, in grado di simulare l'interazione utente su browser reali e di produrre report dettagliati.
- **pytest:** utilizzato per eseguire i test_G di accettazione lato backend, sfruttando fixture che riproducono l'ambiente di produzione e verificando il soddisfacimento dei requisiti_G.
- **GitHub Actions:** le pipeline CI configurate eseguono automaticamente l'intera suite di test_G (inclusi quelli di accettazione) a ogni Pull Request_G e producono log consultabili dal proponente_G.

- **GitHub Issues:** impiegate per registrare feedback_G , richieste di modifica e non conformità emerse durante le sessioni di dimostrazione, garantendo tracciabilità_G e assegnazione $_G$ delle responsabilità.
- **Google Meet/Discord:** piattaforme utilizzate per condurre le sessioni di dimostrazione a distanza con il proponente $_G$, con possibilità di condivisione schermo e registrazione dell'incontro.

3.4 Gestione della configurazione

3.4.1 Processo di gestione della configurazione

Il processo norma il tracciamento e il controllo delle modifiche a tutti gli artefatti $_G$ del ciclo di vita, detti **Configuration Item** (CI).

Ruolo responsabile Amministratore $_G$ di progetto.

Input Artefatto $_G$ da porre sotto controllo di versione; richiesta di modifica a un CI esistente.

Output CI identificato univocamente, versionato $_G$ e tracciabile $_G$; storia delle modifiche registrata.

Obblighi permanenti dell'Amministratore $_G$:

1. **Identificare** univocamente ogni CI e le sue versioni.
2. **Autorizzare** ogni modifica prima che venga integrata: nessun cambiamento entra in *main* senza autorizzazione tracciata $_G$.
3. **Registrare** lo stato di ciascun CI e la storia delle sue modifiche.
4. **Verificare** settimanalmente la completezza e la coerenza della configurazione $_G$.

3.4.2 Attività di gestione della configurazione

3.4.2.1 Regole per l'identificazione dei Configuration Item

Trattare come CI, senza eccezioni, gli artefatti $_G$ seguenti:

- documenti ufficiali (*Norme di Progetto, Analisi dei Requisiti, Piano di Progetto, Piano di Qualifica, Specifica Tecnica*, verbalig $_G$ interni ed esterni);
- codice sorgente del prodotto software $_G$;
- script di build, configurazioni $_G$ di ambiente e pipeline CI/CD;
- diagrammi UML $_G$, risorse grafiche e ogni altro elemento necessario alla realizzazione del sistema $_G$.

Identificare ogni CI mediante il percorso nel repository $_G$ e la storia delle versioni registrata da Git $_G$. È **vietato** mantenere artefatti $_G$ di progetto al di fuori dei repository $_G$ ufficiali.

3.4.2.2 Regole per il versionamento dei documenti

Numerare la versione di ogni documento nel formato $X.Y.Z$. Applicare le regole di incremento seguenti:

Componente	Incrementare quando	Esempio
X (Major)	Il documento entra in una baseline _G (RTB _G , PB _G , CA)	0.9.2 → 1.0.0
Y (Minor)	Si aggiungono sezioni o si modifica la struttura	1.0.3 → 1.1.0
Z (Patch)	Si correggono errori o si adegua la terminologia	1.1.0 → 1.1.1

Tabella 12: Regole di incremento della versione dei documenti

Regole ferree sul versionamento_G:

1. Azzerare le componenti meno significative a ogni incremento di una componente superiore.
2. Registrare **ogni** variazione di versione nel registro delle modifiche_G, indicando versione, data, autore, verificatore_G e descrizione sintetica.
3. Compilare il campo **Approvatore** esclusivamente nelle righe che formalizzano un'effettiva approvazione_G del documento. Lasciarlo vuoto nelle righe di stesura e di verifica_G.
4. Non applicare la convenzione $X.Y.Z$ al codice sorgente: per esso il versionamento_G è gestito dai commit_G Git_G e i rilasci stabili sono marcati con **tag** annotati.

3.4.2.3 Regole per l'uso dei repository

Depositare gli artefatti_G nel repository_G di competenza secondo la tabella seguente:

Repository	Contenuto ammesso
thebitless.github.io	Documentazione di progetto, PDF generati, sito web ufficiale
PoC	Sorgente del <i>Proof of Concept</i> _G e dell'MVP _G , test _G automatici, configurazioni _G di build

Tabella 13: Repository di progetto e contenuto ammesso

3.4.3 Procedure operative

3.4.3.1 Regole per la denominazione dei branch

Creare ogni branch_G nel formato $\langle \text{tipo} \rangle / \langle \text{descrizione-kebab-case} \rangle$. Utilizzare esclusivamente i tipi ammessi in tabella.

Tipo	Creare da	Usare per	Esempio
main	—	Versioni stabili e rilasciate; ogni commit _G è un rilascio marcato con tag	—
develop	main	Integrazione _G delle nuove funzionalità; deve restare sempre compilabile	—
feature/	develop	Nuova funzionalità o miglioramento	feature/autenticazione-utente
fix/	develop	Correzione di un difetto _G non bloccante	fix/rendering-markdown
docs/	develop	Stesura o revisione di documentazione	docs/specifica-tecnica
release/	develop	Stabilizzazione in vista di un rilascio	release/1.0.0
hotfix/	main	Correzione urgente di un difetto _G bloccante in produzione	hotfix/crash-salvataggio

Tabella 14: Tipi di branch ammessi e convenzioni di denominazione

Regole ferree sui branch_G:

1. Scrivere il nome del branch_G in *kebab-case*, in italiano o inglese ma coerentemente all'interno dello stesso repository_G. Non usare spazi, maiuscole o caratteri accentati.
2. È **vietato** eseguire commit_G diretti su **main** e **develop**: ogni modifica passa da una pull request_G.
3. Aprire un branch_G per ogni issue_G: non accorpare in un solo branch_G modifiche riferite a issue_G diverse.
4. Non introdurre nuove funzionalità su un branch_G **release/**: sono ammessi solo interventi di stabilizzazione.
5. Riportare ogni **release/** e **hotfix/** sia in **main** sia in **develop** al termine del lavoro.
6. Eliminare il branch_G dopo il merge_G.

3.4.3.2 Regole per i messaggi di commit

Scrivere ogni messaggio di commit_G nel formato **<tipo>(<ambito>): <descrizione all'infinito>**.

1. Utilizzare i tipi: **feat**, **fix**, **docs**, **refactor**, **test**, **chore**.
2. Mantenere la prima riga entro i **72 caratteri**.
3. Rendere il commit_G **atomico**: un commit_G corrisponde a una sola modifica logica.
4. Non committare codice che non compila o che fa fallire la suite di test_G.

3.4.3.3 Procedura: apertura di una Pull Request

Ruolo responsabile Autore della modifica.

Input Branch_G con le modifiche completate; issue_G collegata; suite di test_G superata in locale.

Output Pull request_G aperta, compilata in ogni campo e assegnata a un Verificatore_G.

Passi da eseguire:

1. Allineare il branch_G al ramo di destinazione e risolvere gli eventuali conflitti in locale.
2. Verificare in locale che formattatore, linter e test_G passino senza segnalazioni.
3. Aprire la pull request_G selezionando branch_G di partenza e branch_G di destinazione (**develop** per le feature, **main** per i rilasci).
4. Assegnare un titolo che riassume lo scopo della modifica.
5. Compilare la descrizione indicando contesto, motivazione e issue_G collegata mediante la sintassi **Closes #<numero>**.
6. Assegnare come revisore un Verificatore_G **diverso dall'autore**.
7. Applicare label_G, progetto GitHub Projects_G e milestone_G di competenza.
8. Attendere l'esito della pipeline di integrazione continua_G prima di richiedere la revisione.

3.4.3.4 Checklist per l'approvazione di una Pull Request

Il Verificatore_G approva la pull request_G **solo se tutte** le voci seguenti risultano soddisfatte. In caso contrario, richiedere modifiche motivando ciascun rilievo.

N.	Voce da verificare
1	La pull request _G è collegata a una issue _G aperta
2	Il nome del branch _G rispetta le convenzioni di denominazione
3	I messaggi di commit _G rispettano il formato definito e sono atomici
4	La pipeline di integrazione continua _G è verde su tutti i passi
5	La copertura _G dei test _G è pari o superiore all'80% dei rami logici
6	Il codice rispetta le norme di codifica (nomi, lingua, formattazione, lunghezza)
7	Nessuna funzione supera la complessità ciclomatica di 10
8	Non sono presenti segreti, chiavi API _G o credenziali nel codice
9	Non è presente codice commentato o non utilizzato
10	La documentazione inline delle funzioni pubbliche è presente e aggiornata
11	I diagrammi UML _G sono allineati alla struttura del codice modificato
12	I documenti impattati sono stati aggiornati e il registro delle modifiche _G è compilato
13	Non sussistono conflitti con il branch _G di destinazione

Tabella 15: Checklist di approvazione di una Pull Request

Regole ferree sul merge_G:

1. È **vietato** approvare la propria pull request_G.
2. È **vietato** effettuare il merge_G prima dell'approvazione_G di almeno un Verificatore_G.
3. Ripetere l'intera checklist a ogni nuovo commit_G aggiunto dopo una richiesta di modifiche.
4. Eliminare il branch_G subito dopo il merge_G.

3.4.3.5 Controllo di configurazione e Change Request

Ruolo responsabile Responsabile_G per la decisione; Analista_G per la valutazione di impatto.

Input Richiesta di modifica a un CI, proveniente dalla proponente_G, dalla committenza_G o dall'interno del gruppo.

Output Change Request approvata_G e pianificata_G, oppure respinta con motivazione documentata nella issue_G.

Passi da eseguire, in questo ordine:

1. **Registrare** la richiesta aprendo una issue_G con label_G **Change request**, indicando obbligatoriamente natura della modifica, urgenza, impatto stimato sui CI esistenti e, se disponibile, proposta di soluzione.
2. **Valutare** la richiesta: stimare fattibilità_G tecnica, impegno in ore, rischi_G introdotti e ricadute su pianificazione_G e budget_G. Registrare la stima nella issue_G.
3. **Decidere** l'approvazione_G o il rifiuto applicando i criteri seguenti: coerenza con gli obiettivi del capitolato_G, disponibilità di ore residue sul ruolo_G competente, compatibilità con le scadenze, priorità espressa dagli stakeholder_G. Motivare la decisione nella issue_G.
4. **Pianificare** la modifica approvata_G: assegnarla a una milestone_G, allocare le ore e aggiornare il *Piano di Progetto*. Se l'impatto eccede il margine disponibile, rinegoziare i tempi con la proponente_G prima di procedere.
5. **Implementare** seguendo il flusso ordinario: branch_G dedicato, commit_G atomici, test_G.
6. **Verificare** che la modifica raggiunga l'obiettivo e non introduca regressioni. Se la modifica tocca una componente critica, eseguire l'intera suite di test_G di regressione prima del merge_G.
7. **Documentare** l'esito nella issue_G e aggiornare il registro delle modifiche_G dei documenti impattati.

Regola ferrea: nessuna modifica a un CI già in baseline_G può essere implementata prima del completamento dei passi 1-4.

3.4.4 Strumenti di supporto

Gestire i Configuration Item esclusivamente con gli strumenti seguenti, secondo le regole indicate.

3.4.4.1 Git

1. Creare un branch_G per ogni issue_G , secondo le convenzioni di denominazione definite sopra.
2. Registrare ogni modifica in un commit_G con autore, data e messaggio conforme al formato definito.
3. Marcare ogni rilascio ufficiale con un **tag** annotato riportante il numero di versione.
4. Non riscrivere la storia di un branch_G già pubblicato: è vietato l'uso di **push --force** su **main** e **develop**.

3.4.4.2 GitHub

1. Registrare in **GitHub Issues** ogni Change Request, difetto $_G$, task $_G$ e attività documentale, assegnando label $_G$, incaricato e milestone $_G$.
2. Aggiornare lo stato delle issue $_G$ sulla board **GitHub Projects** entro la giornata in cui l'attività cambia stato.
3. Configurare **GitHub Actions** per eseguire a ogni pull request $_G$ la compilazione dei documenti, la suite di test $_G$ e il controllo delle convenzioni sui branch_G .
4. Mantenere attiva la **protezione dei branch main e develop**, con divieto di commit $_G$ diretto e obbligo di approvazione $_G$ da parte di un revisore designato.

Verificare mensilmente che le regole di protezione dei branch_G siano ancora attive e coerenti con quanto stabilito in questa sezione. La verifica $_G$ è a carico dell'Amministratore $_G$.

3.5 Risoluzione dei problemi

3.5.1 Processo di risoluzione dei problemi

Ruolo responsabile Chi rileva il problema per la segnalazione; Responsabile $_G$ per l'assegnazione; membro assegnatario per la risoluzione.

Input Comportamento anomalo, difetto $_G$ o non conformità rilevata durante sviluppo $_G$, verifica $_G$ o manutenzione $_G$.

Output Issue $_G$ chiusa con causa radice documentata, azione correttiva applicata ed eventuale azione preventiva registrata.

Obblighi permanenti del processo:

1. Non limitarsi a correggere il sintomo: individuare e documentare la *causa radice* di ogni problema.
2. Definire, per ogni problema ricorrente, un'azione preventiva che ne impedisca il ripetersi.
3. Registrare ogni problema come issue $_G$: un problema non tracciato $_G$ si considera non gestito.
4. Alimentare con i dati raccolti le metriche $_G$ definite nel *Piano di Qualifica*.

3.5.2 Attività di risoluzione dei problemi

3.5.2.1 Gestione dei rischi

Ruolo responsabile Responsabile_G di progetto.

Input Stato di avanzamento; esiti della retrospettiva_G dello sprint_G precedente.

Output Sezione “Analisi dei rischi_G” del *Piano di Progetto* aggiornata.

Passi da eseguire a ogni chiusura di sprint_G:

1. Rivedere l’elenco dei rischi_G censiti e verificare quali si siano effettivamente manifestati nel periodo.
2. Aggiornare, per ciascun rischio_G manifestato, la probabilità di occorrenza e il livello di pericolosità.
3. Valutare l’efficacia delle contromisure applicate; se inefficaci, sostituirle e documentarne la ragione.
4. Censire gli eventuali rischi_G nuovi emersi, assegnando loro un codice secondo la convenzione definita di seguito.
5. Riportare le variazioni nella tabella riassuntiva dei rischi_G del *Piano di Progetto*.

3.5.2.2 Procedura: segnalazione di un problema

Ruolo responsabile Membro che rileva il problema.

Input Comportamento anomalo, difetto_G o non conformità osservata.

Output Issue_G con label_G bug completa di tutti i campi obbligatori e assegnata.

Passi da eseguire:

1. Notificare immediatamente il problema al gruppo sul canale interno.
2. Aprire una issue_G con label_G bug compilando **obbligatoriamente**:
 - descrizione chiara e sintetica del problema;
 - passi esatti per riprodurre il comportamento anomalo;
 - comportamento atteso e comportamento osservato, messi a confronto;
 - ambiente in cui si è verificato (versione del software_G, sistema_G operativo, configurazione_G);
 - allegati utili alla diagnosi (log, screenshot, file di configurazione_G).
3. Assegnare la issue_G al membro più competente sulla componente interessata.

Regola ferrea: una segnalazione priva dei passi di riproduzione è respinta e rinviata al segnalante per il completamento.

3.5.3 Procedure operative

3.5.3.1 Codifica dei rischi

Per garantire univocità e tracciabilità_G, ogni rischio_G identificato nel *Piano di Progetto*_G viene codificato secondo la seguente convenzione:

R[Tipologia]-[Codice] : Nome associato al rischio_G
dove:

- **Tipologia:** indica la natura del rischio_G. I valori ammessi sono:
 - T – Rischi legati all'utilizzo delle tecnologie (incompatibilità, obsolescenza, difficoltà di apprendimento, mancanza di documentazione);
 - O – Rischi legati all'organizzazione del gruppo (conflitti, indisponibilità di membri, scarsa comunicazione, sovraccarico di lavoro);
 - P – Rischi legati al prodotto (requisiti_G instabili, complessità architetturale, difettosità latente, performance inadeguate).
- **Codice:** valore numerico incrementale_G che identifica in modo univoco il rischio_G all'interno della sua tipologia (es. RT-1, RO-3, RP-2).

Questa codifica consente di riferirsi ai rischi in modo non ambiguo in tutta la documentazione di progetto e di monitorarne l'evoluzione nel tempo.

3.5.3.2 Procedura di segnalazione e gestione dei problemi

Quando un membro del gruppo identifica un problema, deve seguire la procedura operativa descritta di seguito, che garantisce la presa in carico tempestiva e la tracciabilità_G completa.

1. **Notifica immediata:** il problema viene comunicato al gruppo attraverso il canale di comunicazione dedicato su Discord_G (canale **problemi** o simile), descrivendo sinteticamente l'anomalia e la componente interessata.
2. **Apertura della Issue_G:** il membro che ha rilevato il problema apre una Issue_G su GitHub_G, assegnando l'etichetta **bug** e compilando il template di segnalazione con tutte le informazioni richieste (descrizione, passi per riprodurre, comportamento atteso e osservato, ambiente, allegati).
3. **Valutazione e assegnazione:** il Responsabile_G o l'Amministratore_G valuta la Issue_G, ne stima la priorità (alta, media, bassa) e la assegna al membro del gruppo più idoneo per la risoluzione. Se il problema è critico e blocca lo sviluppo_G, può essere attivata una procedura di hotfix secondo il workflow_G Gitflow.
4. **Analisi e risoluzione:** l'assegnatario analizza la causa del problema, propone una soluzione e la implementa su un branch_G dedicato. Durante l'implementazione vengono aggiunti o aggiornati i test_G automatici per coprire il caso specifico e prevenire regressioni future.
5. **Verifica:** la soluzione viene sottoposta a verifica_G tramite Pull Request_G, seguendo le procedure descritte in §3.2. Il verificatore_G controlla che il problema sia effettivamente risolto e che i test_G associati siano superati.

6. **Chiusura:** una volta verificata e mergiata la soluzione, la $Issue_G$ viene chiusa. Nel commento di chiusura si riporta un riepilogo della soluzione adottata e delle eventuali azioni preventive intraprese.
7. **Capitalizzazione:** se il problema è ritenuto significativo, viene discusso durante la successiva riunione di gruppo per condividere le lezioni apprese e valutare l'aggiornamento della lista dei rischi o delle procedure.

Questa procedura assicura che ogni problema sia gestito in modo disciplinato, documentato e verificabile, in linea con i principi di miglioramento continuo.

3.5.4 Strumenti di supporto

Per la gestione del processo di risoluzione dei problemi, il gruppo si avvale dei seguenti strumenti:

- **GitHub Issues:** $sistema_G$ di ticketing $_G$ utilizzato per la segnalazione, il tracciamento e la gestione di tutti i problemi rilevati. Ogni problema è rappresentato da una $Issue_G$ con etichetta *bug*, a cui vengono associati $label_G$, $milestone_G$ e assegnatari. La piattaforma consente di mantenere una cronologia completa delle discussioni, delle soluzioni proposte e delle decisioni prese.
- **GitHub Projects:** la $dashboard_G$ Kanban consente di monitorare visivamente lo stato delle $Issue_G$ relative ai problemi, spostandole tra le colonne “Da fare”, “In corso”, “In revisione” e “Completato”. Questa vista facilita il coordinamento del gruppo e la prioritizzazione degli interventi.
- **Discord:** il canale di comunicazione dedicato consente di notificare tempestivamente i problemi critici e di discutere in tempo reale le possibili soluzioni, integrandosi con le segnalazioni formali registrate su $GitHub_G$.
- **Git:** il $sistema_G$ di versionamento distribuito permette di isolare la risoluzione di ogni problema su un $branch_G$ dedicato, di tracciare le modifiche e di associare ogni $commit_G$ alla $Issue_G$ di riferimento, garantendo la piena tracciabilità $_G$ tra codice e segnalazioni.

4 Processi organizzativi

I processi organizzativi governano pianificazione $_G$, coordinamento, miglioramento e formazione del gruppo. Le procedure di questa sezione sono vincolanti al pari di quelle dei processi primari e di supporto.

4.1 Gestione dei Processi

4.1.1 Processo di gestione dei processi

Ruolo responsabile Responsabile $_G$ di progetto.

Input Obiettivi di $sprint_G$; disponibilità dichiarata dai membri; esiti della retrospettiva $_G$ precedente.

Output *Piano di Progetto* aggiornato con pianificazione $_G$, preventivo $_G$, consuntivo $_G$ e analisi dei rischi $_G$.

Presidiare le cinque aree seguenti, con la cadenza indicata:

Area	Azione da compiere	Cadenza
Definizione dei processi	Documentare procedure e modelli di esecuzione nelle presenti norme	A ogni variazione del <i>Way of Working_G</i>
Pianificazione _G e monitoraggio	Redigere il piano di sprint _G e confrontare avanzamento reale e previsto	Apertura e chiusura di ogni sprint _G
Valutazione e miglioramento	Rilevare le metriche _G di processo e definire azioni correttive	Chiusura di ogni sprint _G
Formazione e competenze	Verificare che ogni membro disponga delle competenze per il ruolo _G assegnato	Apertura di ogni sprint _G
Gestione dei rischi _G	Aggiornare probabilità, pericolosità e contromisure	Chiusura di ogni sprint _G

Tabella 16: Aree della gestione dei processi e cadenza di presidio

4.1.2 Attività di gestione

4.1.2.1 Pianificazione

Ruolo responsabile Responsabile_G di progetto.

Input Obiettivi dello sprint_G; ore residue per ruolo_G; disponibilità dichiarata dai membri.

Output Sezione di pianificazione_G e preventivo_G dello sprint_G nel *Piano di Progetto*; issue_G create e assegnate.

Passi da eseguire all'apertura di ogni sprint_G:

1. Raccogliere la disponibilità oraria dichiarata da ciascun membro per il periodo.
2. Definire gli obiettivi dello sprint_G e le attività necessarie a raggiungerli.
3. Assegnare i ruoli_G secondo la rotazione_G, verificando che ogni membro ricopra tutti i ruoli_G previsti almeno una volta nel corso del progetto.

4. Stimare le ore per ruolo_G, applicando un margine esplicito alle attività prive di precedenti storici.
5. Verificare che le ore stimate non eccedano il monte ore residuo per ciascun ruolo_G; in caso contrario, ridurre lo scopo o ridistribuire.
6. Aprire le issue_G corrispondenti e assegnarle ai membri.
7. Registrare pianificazione_G e preventivo_G nel *Piano di Progetto*.

Contenuto obbligatorio del piano: attività da svolgere, risorse necessarie, tempistiche per attività, tecnologie e strumenti impiegati, personale assegnato a ogni compito.

4.1.2.2 Assegnazione dei ruoli e responsabilità

Durante il progetto, i membri del gruppo assumono sei ruoli distinti, ciascuno con responsabilità e compiti specifici. I ruoli sono descritti di seguito.

- **Responsabile**

È la figura che coordina il gruppo e funge da punto di riferimento per il proponente_G e per il team. Gestisce le relazioni esterne, pianifica le attività (definendo cosa fare, quando e con quali priorità), valuta i rischi, controlla i progressi, gestisce le risorse umane e approva formalmente la documentazione.

- **Amministratore**

Si occupa del controllo e dell'amministrazione dell'ambiente di lavoro. Gestisce il versionamento della documentazione e la configurazione del prodotto, redige e attua le norme e le procedure per la qualità, amministra le infrastrutture e i servizi a supporto dei processi.

- **Analista**

Possiede competenze avanzate nell'analisi dei requisiti_G e nella comprensione del dominio_G applicativo. Il suo compito è identificare, documentare e comprendere a fondo le esigenze del progetto, traducendole in requisiti_G chiari e dettagliati. Studia i bisogni espliciti e impliciti e redige il documento *Analisi dei Requisiti*_G.

- **Progettista**

È il riferimento per le scelte progettuali e tecnologiche, a partire dal lavoro dell'Analista_G. Definisce l'architettura del prodotto, prende decisioni tecniche per garantire affidabilità, efficienza e conformità ai requisiti_G, e redige il *Piano di Qualifica*_G.

- **Programmatore**

È incaricato della scrittura del codice. Implementa le componenti dell'architettura seguendo le specifiche tecniche, realizza codice manutenibile, sviluppa gli strumenti per la verifica_G e la validazione_G del codice e redige il *Manuale Utente*.

- **Verificatore**

Ha la responsabilità di ispezionare il lavoro svolto dagli altri membri del team per assicurarne la qualità e la conformità alle attese. Verifica_G che i prodotti rispettino le *Norme di Progetto* e i Requisiti Funzionali_G e di qualità, segnalando tempestivamente eventuali errori o non conformità.

4.1.2.3 Ticketing e tracciamento delle attività

Per gestire in modo trasparente e ordinato le attività, il gruppo utilizza GitHub_G come sistema_G di tracciamento delle issue_G (ITS). L'Amministratore_G crea e assegna le issue_G in base alle attività individuate dal Responsabile_G, garantendo che ogni compito abbia un assegnatario chiaro e una scadenza definita.

Ogni issue_G deve contenere:

- un titolo conciso e descrittivo;
- una descrizione testuale dettagliata o, in alternativa, una lista di cose da fare;
- il nome del verificatore_G, indicato nell'ultima riga della descrizione nel formato "Verificatore_G: Nome Cognome";
- l'assegnatario (uno o più membri incaricati dello svolgimento);
- una data di scadenza;
- etichette (*label*) per categorizzare l'issue_G (es. Verbale, Documents, Develop, Bug_G, Feature);
- etichette aggiuntive per associare l'issue_G al Configuration Item corrispondente: NdP, PdQ, PdP, AdR, PoC_G, Gls;
- la milestone_G di riferimento;
- il progetto GitHub Projects_G a cui l'issue_G appartiene;
- l'eventuale branch_G e la Pull Request_G collegati (se lo sviluppo_G è in corso).

L'utilizzo delle dashboard_G di progetto (Dashboard_G e Roadmap) consente a ogni membro del gruppo di monitorare in tempo reale lo stato di avanzamento delle attività, visualizzando le issue_G suddivise per fase e per scadenza.

4.1.3 Procedure operative

4.1.3.1 Procedura di pianificazione

La pianificazione segue un flusso strutturato, condotto dal Responsabile_G in collaborazione con l'Amministratore_G:

1. **Identificazione delle attività:** a partire dagli obiettivi di periodo e dagli impegni presi con il proponente_G, il Responsabile_G individua tutte le attività da svolgere.
2. **Stima dell'impegno e delle risorse:** per ogni attività viene stimato il tempo necessario e vengono identificate le competenze richieste.
3. **Assegnazione dei ruoli:** il Responsabile_G assegna i ruoli ai membri del gruppo, garantendo la rotazione e il bilanciamento dei carichi di lavoro.
4. **Redazione del Piano di Progetto:** le attività, le stime e le assegnazioni vengono formalizzate nel *Piano di Progetto*_G, che viene versionato e sottoposto a verifica_G.
5. **Creazione delle issue:** l'Amministratore_G traduce le attività pianificate in issue_G su GitHub_G, assegnandole ai membri designati e impostando le scadenze.

6. **Monitoraggio e aggiornamento:** durante l'esecuzione, il Responsabile_G verifica_G lo stato di avanzamento e, se necessario, aggiorna il piano, riportando le modifiche nel documento.

4.1.3.2 Procedura di gestione del ciclo di vita delle issue

Il ciclo di vita di una issue_G, dalla creazione alla chiusura, segue una procedura operativa standard:

1. L'Amministratore_G crea la issue_G su GitHub_G, compilandone tutti gli attributi e assegnandola al membro responsabile_G.
2. L'Amministratore_G sposta la issue_G nella colonna "To Do" della dashboard_G di progetto.
3. L'assegnatario crea un branch_G dedicato su GitHub_G, seguendo le convenzioni di nomenclatura.
4. Quando l'assegnatario prende in carico il lavoro, sposta la issue_G dalla colonna "To Do" a "In Progress".
5. Completata l'attività, l'assegnatario apre una Pull Request_G verso il branch_G di destinazione.
6. L'assegnatario sposta la issue_G nella colonna "Da revisionare" della dashboard_G.
7. Il Verificatore_G designato esamina le modifiche applicando le procedure di verifica_G.
8. Se la verifica_G ha esito positivo, il Verificatore_G approva la Pull Request_G; la issue_G viene automaticamente chiusa e spostata nella colonna "Done" della dashboard_G. In caso di esito negativo, il Verificatore_G richiede modifiche e la issue_G rimane in "Da revisionare" fino alla risoluzione.

4.1.4 Strumenti di supporto

- **GitHub**

Piattaforma centrale per il tracciamento e la gestione delle attività. Le funzionalità impiegate includono:

- **GitHub Issues:** sistema_G di ticketing_G per creare, assegnare e monitorare i compiti. Ogni issue_G è collegabile a milestone_G, label, branch_G e Pull Request_G.
- **GitHub Projects:** dashboard_G Kanban per visualizzare lo stato delle issue_G (To Do, In Progress, Da revisionare, Done) e per avere una panoramica immediata dell'avanzamento.
- **Roadmap:** vista temporale che mostra la distribuzione delle issue_G nel tempo, utile per il monitoraggio delle scadenze.

- **Google Calendar**

Utilizzato per la pianificazione temporale delle attività. Consente di fissare scadenze, organizzare riunioni e sincronizzare gli impegni di tutto il gruppo, inviando promemoria automatici.

- **Discord**

Piattaforma di comunicazione istantanea impiegata per il coordinamento quotidiano. I canali tematici permettono di discutere rapidamente lo stato delle attività, assegnare compiti urgenti e condividere aggiornamenti in tempo reale.

4.2 Coordinamento

4.2.1 Processo di coordinamento

Ruolo responsabile Responsabile_G di progetto.

Input Stato di avanzamento delle attività; disponibilità dichiarata dai membri; necessità di confronto interno o esterno.

Output Riunioni svolte e verbalizzate; decisioni tracciate_G come issue_G; membri allineati sullo stato del progetto.

Obblighi permanenti del Responsabile_G:

1. Convocare la riunione interna settimanale e verificarne il raggiungimento del quorum.
2. Accertare che ogni decisione assunta sia verbalizzata e associata a un incaricato.
3. Verificare che ogni membro conosca le attività a lui assegnate per lo sprint_G in corso.
4. Intervenire entro 48 ore su ogni situazione di stallo comunicativo segnalata da un membro.

4.2.2 Attività di coordinamento

Selezionare il canale di comunicazione secondo la matrice seguente. È vietato trattare una comunicazione su un canale diverso da quello previsto.

Tipo	Destinatario	Canale	Usare per
Sincrona	Interno	Discord	Riunioni settimanali, confronto tecnico, sblocco di criticità
Sincrona	Esterno	Google Meet	Incontri SAL con la proponente _G , previa pianificazione _G
Asincrona	Interno	WhatsApp	Comunicazioni logistiche e avvisi rapidi
Asincrona	Interno	GitHub Issues	Assegnazioni, decisioni tecniche, segnalazione di difetti _G
Asincrona	Esterno	Google Mail	Corrispondenza ufficiale con la proponente _G

Tabella 17: Matrice di selezione del canale di comunicazione

Regola ferrea: riportare su GitHub Issues ogni decisione assunta su un canale sincrono o su WhatsApp che incida su requisiti_G, scadenze o assegnazioni. Una decisione non tracciata_G non è vincolante.

4.2.2.1 Riunioni interne

Regole ferree sulle riunioni interne:

1. Convocare la riunione ordinaria a cadenza **settimanale**, indicativamente il martedì, in presenza o da remoto.
2. Rinviare la riunione se è assente la maggioranza dei membri: senza quorum non si assumono decisioni vincolanti.

3. Convocare riunioni straordinarie al presentarsi di criticità bloccanti.
4. Affidare la moderazione e la redazione del verbale_G interno al **Responsabile_G**, in quanto unico documento che attesta decisioni e impegni assunti.
5. Affidare all'**Amministratore_G** il supporto organizzativo: ordine del giorno, convocazione, registrazione delle presenze, apertura delle issue_G al termine.

4.2.2.2 Riunioni esterne

Regole ferree sulle riunioni esterne:

1. Affidare al Responsabile_G convocazione, definizione dell'ordine del giorno e conduzione dell'incontro.
2. Garantire la presenza di tutti i membri, salvo impedimenti inderogabili.
3. Comunicare tempestivamente alla proponente_G l'assenza di un membro e, se la sua presenza è necessaria, richiedere il rinvio.
4. Affidare la redazione del verbale_G esterno al Responsabile_G e sottoporlo all'approvazione_G della controparte prima del consolidamento.

4.2.3 Procedure operative

4.2.3.1 Procedura per le riunioni interne

1. **Convocazione:** il Responsabile_G, coadiuvato dall'Amministratore_G, comunica data, ora e modalità dell'incontro (presenza o telematico) attraverso i canali di comunicazione interni, con un preavviso di almeno 24 ore.
2. **Ordine del giorno:** prima della riunione, il Responsabile_G e l'Amministratore_G predispongono un elenco dei punti da discutere, tenendo conto delle segnalazioni ricevute dai membri e delle scadenze imminenti.
3. **Svolgimento:** il Responsabile_G modera la discussione, assicurando che ogni punto venga affrontato e che tutti i partecipanti abbiano modo di esprimersi. L'Amministratore_G registra le presenze e prende appunti sulle decisioni e sui task_G emergenti.
4. **Verbalizzazione:** al termine della riunione, il Responsabile_G redige il verbale interno secondo la struttura standard definita nelle Norme di Progetto_G, riportando ordine del giorno, sintesi delle discussioni, decisioni prese e attività da svolgere con relative scadenze e assegnatari.
5. **Follow-up:** entro 24 ore dalla riunione, l'Amministratore_G crea o aggiorna le issue_G sull'ITS corrispondenti ai nuovi task_G e sposta le issue_G esistenti nelle colonne appropriate della dashboard_G di progetto. Il verbale viene condiviso in bozza sui canali interni per eventuali commenti, quindi consolidato.

4.2.3.2 Procedura per le riunioni esterne

1. **Convocazione:** il Responsabile_G contatta il proponente_G via email, proponendo date e orari compatibili con gli impegni del gruppo. Ottenuta conferma, crea un evento sul calendario condiviso e notifica il team.
2. **Preparazione:** il Responsabile_G, sentiti i membri del gruppo, definisce l'ordine del giorno e prepara il materiale da presentare (stato avanzamento lavori, dimostrazioni, quesiti).
3. **Svolgimento:** durante la riunione, il Responsabile_G conduce la discussione. I membri del team intervengono per illustrare le parti di propria competenza. Le decisioni e gli impegni presi vengono annotati dal Responsabile_G.
4. **Verbalizzazione:** il Responsabile_G redige il verbale esterno entro 48 ore dall'incontro, utilizzando il template ufficiale. Il verbale viene sottoposto all'approvazione_G del proponente_G prima di essere consolidato.
5. **Registrazione e azioni:** i task_G e gli impegni emersi vengono tradotti in issue_G dall'Amministratore_G, collegandoli al verbale e inserendoli nella milestone_G di riferimento. Il verbale approvato viene archiviato nel repository_G documentale.

4.2.4 Strumenti di supporto

Impiegare gli strumenti di coordinamento secondo la matrice dei canali definita sopra. Regole operative aggiuntive:

1. Inserire in **Google Calendar** ogni riunione, scadenza e appuntamento con la proponente_G, con promemoria automatico attivo.
2. Conservare lo storico delle conversazioni su **Discord**: non eliminare i canali di progetto.
3. Collegare su **GitHub** ogni issue_G e pull request_G al verbale_G della riunione in cui la relativa decisione è stata assunta.
4. Inviare da **Gmail** convocazioni, verbali_G e documentazione destinati alla proponente_G, conservandone copia nel thread di riferimento.

4.3 Miglioramento

4.3.1 Processo di miglioramento

Secondo lo standard ISO/IEC 12207:1995, il processo di miglioramento nel ciclo di vita del software ha lo scopo di stabilire, misurare, controllare e migliorare i processi che lo compongono. Si tratta di un processo continuo e trasversale, che non si esaurisce in una singola fase ma accompagna l'intero progetto, garantendo che l'organizzazione impari dall'esperienza e aumenti progressivamente la propria efficacia.

Le finalità del processo di miglioramento sono:

- **valutare** costantemente le prestazioni dei processi adottati;
- **identificare** le aree che presentano inefficienze, colli di bottiglia o risultati insoddisfacenti;
- **pianificare e attuare** azioni correttive e preventive mirate;

- **verificare** l'efficacia delle azioni intraprese e consolidare i miglioramenti ottenuti.

Il processo si basa su un ciclo iterativo di analisi e miglioramento, che trae alimento dalle retrospettive periodiche e dalla documentazione prodotta durante le attività di progetto. In questo modo, il gruppo trasforma le criticità e gli errori in opportunità di crescita, affinando progressivamente metodi, strumenti e dinamiche di lavoro.

4.3.2 Attività di miglioramento

4.3.2.1 Analisi

Ruolo responsabile Responsabile_G come conduttore; Amministratore_G come verbalizzante.

Input Stato delle issue_G; esiti delle verifiche_G; scostamenti_G orari del periodo; criticità segnalate.

Output Elenco di azioni correttive concordate, ciascuna con incaricato e scadenza.

Condurre la retrospettiva_G **all'inizio di ogni riunione interna**, coinvolgendo tutti i membri e trattando obbligatoriamente i quattro punti seguenti:

1. ciò che ha funzionato e va preservato;
2. ciò che ha creato ostacoli, rallentamenti o risultati inferiori alle attese;
3. le cause profonde dei problemi emersi, distinguendo i fattori contingenti dalle criticità strutturali;
4. le soluzioni da adottare nel periodo successivo.

Regola ferrea: non chiudere la retrospettiva_G senza almeno un'azione correttiva formalizzata per ciascuna criticità strutturale emersa.

4.3.2.2 Miglioramento

Ruolo responsabile Incaricato designato per ciascuna azione correttiva.

Input Azione correttiva concordata in retrospettiva_G.

Output Azione attuata; esito valutato e documentato nel verbale_G interno successivo.

Passi da eseguire:

1. Registrare ogni azione correttiva come issue_G con label_G **improvement**, indicando descrizione, incaricato e scadenza.
2. Attuare l'azione entro la scadenza concordata, aggiornando lo stato della issue_G.
3. Valutare l'efficacia dell'azione nella retrospettiva_G successiva, classificandone l'esito come: completata con successo, parzialmente efficace, inefficace, non completata.
4. Documentare l'esito nella sezione "Revisione del periodo precedente" del verbale_G interno.
5. Se l'azione si rivela stabilmente efficace, recepirla nelle presenti Norme di Progetto_G come procedura permanente.

4.3.3 Procedure operative

4.3.3.1 Procedura di retrospettiva e miglioramento

1. **Preparazione:** prima della riunione, il Responsabile_G e l'Amministratore_G raccolgono i dati rilevanti sull'andamento dell'ultimo periodo (stato delle issue_G, esiti delle verifiche, eventuali criticità segnalate) per fornire al team una base oggettiva su cui discutere.
2. **Retrospettiva:** all'inizio della riunione interna, il Responsabile_G guida il team in una discussione strutturata, invitando ciascun membro a esprimere osservazioni e proposte. L'Amministratore_G annota i punti salienti e le azioni correttive proposte.
3. **Formalizzazione delle azioni:** al termine della discussione, il team concorda una lista di azioni correttive. Per ciascuna vengono definiti: descrizione, responsabile_G dell'implementazione, scadenza. Le azioni vengono registrate come issue_G nell'ITS con un'etichetta dedicata (ad esempio **improvement**).
4. **Implementazione:** i responsabili attuano le azioni correttive entro la scadenza concordata, aggiornando lo stato delle issue_G man mano che il lavoro procede.
5. **Verifica e documentazione:** nella riunione successiva, il team esamina lo stato di ogni azione. L'esito (completata con successo, parzialmente efficace, inefficace, non completata) viene discusso e riportato nella sezione "Revisione del periodo precedente" del verbale interno. Le issue_G relative alle azioni completate vengono chiuse; quelle che necessitano di ulteriori interventi rimangono aperte e vengono riprogrammate.
6. **Capitalizzazione:** le soluzioni che si rivelano particolarmente efficaci vengono valutate per essere integrate stabilmente nelle procedure di progetto. Se opportuno, le Norme di Progetto_G vengono aggiornate per riflettere il nuovo assetto processuale.

4.3.4 Strumenti di supporto

- **GitHub Issues:** le azioni correttive vengono tracciate come issue_G con l'etichetta **improvement**, consentendo di monitorarne lo stato, assegnare responsabili e fissare scadenze. L'integrazione_G con la dashboard_G di progetto permette di visualizzarle insieme a tutte le altre attività.
- **GitHub Projects:** la dashboard_G Kanban offre una vista immediata sullo stato delle azioni di miglioramento (To Do, In Progress, Done), facilitando il follow-up durante le riunioni successive.
- **Discord:** durante le riunioni, i canali testuali possono essere utilizzati per raccogliere in tempo reale spunti e osservazioni dai membri, che l'Amministratore_G può poi sistematizzare nelle issue_G.

4.4 Formazione

4.4.1 Processo di formazione

Ruolo responsabile Responsabile_G per la rilevazione dei fabbisogni; ciascun membro per la propria formazione.

Input Ruoli_G assegnati per lo sprint_G; tecnologie adottate; lacune rilevate nella retrospettiva_G.

Output Membri in grado di operare sul ruolo_G assegnato; materiale formativo condiviso nel repository_G.

Regole ferree del processo di formazione:

1. Verificare, all'apertura di ogni sprint_G, che ciascun membro disponga delle competenze richieste dal ruolo_G assegnato. In caso di lacuna, pianificare_G le ore di formazione **prima** di assegnare attività produttive su quel ruolo_G.
2. Affiancare il membro entrante al membro uscente a ogni rotazione_G dei ruoli_G, con un passaggio di consegne documentato.
3. Documentare nel repository_G ogni conoscenza acquisita che sia necessaria ad altri: una competenza detenuta da un solo membro costituisce un rischio_G organizzativo da segnalare.
4. Richiedere alla proponente_G sessioni formative sulle tecnologie di progetto quando la formazione autonoma non risulti sufficiente.
5. Non rendicontare le ore di formazione come ore produttive.

4.4.2 Attività di formazione

Le attività di formazione si articolano su due livelli: individuale e di gruppo. Entrambe concorrono a costruire un bagaglio di competenze omogeneo e a garantire che nessun membro rimanga escluso dalla piena operatività.

4.4.2.1 Formazione individuale

Ciascun membro del team si impegna in un percorso di autoformazione finalizzato a colmare le lacune personali e a prepararsi per il ruolo che andrà a ricoprire. Questa attività è particolarmente rilevante in fase di avvio del progetto e in corrispondenza dei cambi di ruolo. L'autoformazione si basa su:

- studio di documentazione tecnica, guide e tutorial messi a disposizione dal gruppo o indicati dal proponente_G;
- sperimentazione pratica sugli ambienti di sviluppo_G e sugli strumenti di progetto;
- confronto con i membri che hanno già ricoperto il ruolo in precedenza.

Un momento cruciale della formazione individuale è rappresentato dal **passaggio di consegne** che avviene durante la rotazione dei ruoli_G. Ogni membro, prima di lasciare un ruolo, organizza un incontro con il successivo assegnatario per trasferire le conoscenze pratiche accumulate, le procedure consolidate e le criticità note. Simmetricamente, prima di assumere un nuovo ruolo, il membro si confronta con chi lo ha preceduto per apprendere le competenze richieste, ricevendo indicazioni mirate e materiale di supporto.

4.4.2.2 Formazione di gruppo

La formazione di gruppo è costituita da sessioni formative condotte dal proponente_G. Questi incontri sono specificamente orientati al trasferimento di competenze sulle tecnologie impiegate nel progetto, con particolare riferimento a:

- strumenti e framework_G di sviluppo $_G$ adottati;
- architettura del sistema $_G$ e scelte progettuali;
- procedure operative definite dal proponente $_G$ per il dominio $_G$ applicativo.

La partecipazione del team a tali sessioni è obbligatoria, poiché esse costituiscono un momento insostituibile per allineare le conoscenze di tutti i membri e per ottenere risposte dirette a dubbi e quesiti.

Durante le sessioni, i membri del team sono invitati a prendere appunti e a condividere successivamente i punti salienti sul repository $_G$ documentale, in modo da creare una base di conoscenza riutilizzabile in futuro e accessibile anche a chi non ha potuto partecipare.

4.4.3 Procedure operative

4.4.3.1 Procedura per il passaggio di consegne (rotazione dei ruoli)

1. **Pianificazione dell'incontro:** il Responsabile $_G$, in fase di pianificazione delle attività, individua le coppie di membri coinvolte nella rotazione e fissa un incontro di passaggio consegne prima dell'inizio del nuovo periodo.
2. **Preparazione del materiale:** il membro uscente prepara una sintesi delle attività svolte, delle procedure apprese e delle criticità incontrate. Può avvalersi di documentazione già esistente o produrre appunti specifici.
3. **Svolgimento dell'incontro:** l'incontro si svolge preferibilmente in modalità sincrona, utilizzando gli strumenti di comunicazione del gruppo. Il membro uscente illustra al subentrante le informazioni chiave e risponde ai quesiti. Il membro entrante prende nota degli aspetti più rilevanti.
4. **Condivisione e archiviazione:** gli appunti prodotti durante l'incontro vengono caricati in una sezione dedicata del repository $_G$ documentale o in uno spazio condiviso, in modo che restino disponibili per consultazioni future.
5. **Verifica dell'efficacia:** dopo un periodo di rodaggio, il Responsabile $_G$ verifica $_G$ informalmente con il nuovo assegnatario che il passaggio di consegne sia stato sufficiente e, in caso contrario, dispone ulteriori momenti di affiancamento.

4.4.3.2 Procedura per le sessioni formative con la proponente

1. **Convocazione:** il proponente $_G$ comunica data, ora e modalità della sessione formativa. Il Responsabile $_G$ del gruppo provvede a informare tempestivamente tutti i membri e a creare un evento sul calendario condiviso.
2. **Preparazione:** i membri del team, se necessario, studiano materiale propedeutico suggerito dal proponente $_G$ per affrontare la sessione con un livello minimo di conoscenze pregresse.
3. **Partecipazione:** tutti i membri partecipano alla sessione. Durante l'incontro, è possibile porre domande e richiedere chiarimenti. Un membro designato (solitamente l'Amministratore $_G$) prende appunti dettagliati.

4. **Rielaborazione e condivisione:** entro 48 ore dalla sessione, gli appunti vengono rielaborati in un formato leggibile e condivisi con il team tramite il repository_G documentale o altra piattaforma concordata. In questo modo il contenuto formativo diventa patrimonio comune del gruppo.
5. **Follow-up:** eventuali dubbi residui vengono raccolti e, se necessario, sottoposti al proponente_G via email per un chiarimento successivo.

4.4.4 Strumenti di supporto

- **Google Meet:** piattaforma utilizzata per le sessioni formative sincrone con il proponente_G, in virtù della facilità di accesso e della possibilità di condividere schermo e materiali.
- **Discord:** i canali tematici interni possono ospitare discussioni di follow-up alle sessioni formative, domande e risposte rapide tra membri, e momenti di formazione informale tra pari.
- **GitHub:** il repository_G documentale funge da archivio permanente per gli appunti delle sessioni formative e per il materiale di passaggio consegne. La possibilità di versionare i documenti garantisce che la conoscenza accumulata non vada persa e possa essere aggiornata nel tempo.
- **Google Calendar:** utilizzato per fissare gli appuntamenti di passaggio consegne e per ricordare le sessioni formative, assicurando che tutti i membri siano informati per tempo e possano organizzare i propri impegni.
- **WhatsApp:** per comunicazioni rapide e informali relative alla formazione, come la condivisione di link a risorse utili o la richiesta di chiarimenti su argomenti specifici.

5 Metriche e standard per la Qualità

Il gruppo adotta lo standard **ISO/IEC 12207** quale riferimento per la definizione delle caratteristiche che determinano la qualità del prodotto software. Tale standard individua sei macro-caratteristiche principali, ciascuna delle quali è declinata in una serie di attributi misurabili, utili a valutare in modo oggettivo il livello qualitativo raggiunto dal sistema_G.

5.1 Funzionalità

La funzionalità esprime la capacità del prodotto di soddisfare i bisogni espliciti e impliciti degli stakeholder_G, fornendo l'insieme di funzioni necessarie per il raggiungimento degli obiettivi prefissati. Gli attributi che concorrono a definirla sono:

- **Appropriatezza:** indica se il prodotto mette a disposizione le funzioni necessarie a svolgere i compiti previsti in fase di analisi.
- **Accuratezza:** misura la capacità del sistema_G di fornire risultati corretti e privi di errori non desiderati.
- **Interoperabilità:** esprime l'attitudine del prodotto a interagire con altri sistemi, scambiando dati e servizi secondo protocolli e formati definiti.

- **Conformità:** verifica_G l'aderenza del prodotto a standard, leggi o regolamenti pertinenti al dominio_G applicativo.
- **Sicurezza:** riguarda la protezione delle informazioni e delle risorse del sistema_G, impedendo accessi non autorizzati e preservando la riservatezza, l'integrità e la disponibilità dei dati.

5.2 Affidabilità

L'affidabilità rappresenta la capacità del software di mantenere un livello di prestazioni accettabile nel tempo e in condizioni operative specificate. È scomposta nei seguenti attributi:

- **Maturità:** capacità del sistema_G di evitare malfunzionamenti dovuti a difetti interni, specialmente in situazioni di carico normale.
- **Tolleranza ai guasti:** attitudine a mantenere un comportamento corretto anche in presenza di anomalie o errori di componenti con cui il sistema_G interagisce.
- **Recuperabilità:** capacità di ripristinare lo stato operativo e i dati dopo un'interruzione, minimizzando la perdita di informazioni.

5.3 Efficienza

L'efficienza concerne il rapporto tra le prestazioni del prodotto e le risorse impiegate per ottenerle. I principali attributi sono:

- **Tempo di risposta:** tempo necessario al sistema_G per elaborare una richiesta e restituire un risultato all'utente o a un altro sistema_G.
- **Utilizzo delle risorse:** misura dell'impiego di memoria, processore, banda di rete e altre risorse durante l'esecuzione delle funzioni previste.

5.4 Usabilità

L'usabilità definisce il grado di facilità con cui gli utenti finali possono interagire con il prodotto. Gli attributi che la caratterizzano sono:

- **Comprensibilità:** chiarezza con cui l'interfaccia e le funzionalità sono presentate all'utente, rendendo immediatamente evidente il loro scopo.
- **Apprendibilità:** tempo e sforzo necessari a un utente inesperto per acquisire familiarità con le operazioni principali del sistema_G.
- **Operatività:** disponibilità di strumenti che agevolano l'esecuzione dei compiti, come scorciatoie, menu contestuali e automazioni.

5.5 Manutenibilità

La manutenibilità_G esprime la facilità con cui il prodotto può essere modificato, corretto o esteso nel tempo. I suoi attributi sono:

- **Analizzabilità:** facilità con cui è possibile esaminare il codice sorgente e la documentazione per individuare le cause di un malfunzionamento o di un comportamento anomalo.

- **Modificabilità:** misura dello sforzo richiesto per apportare cambiamenti al software, sia a livello di singoli moduli sia a livello architetturale.
- **Stabilità:** capacità del sistema_G di non introdurre effetti collaterali indesiderati a seguito di modifiche, mantenendo invariato il comportamento delle parti non interessate.

5.6 Portabilità

La portabilità descrive l'attitudine del prodotto a essere trasferito da un ambiente di esecuzione a un altro, sia esso hardware o software. Gli attributi che la compongono sono:

- **Adattabilità:** facilità con cui il software può essere reso operativo su piattaforme diverse da quella originariamente prevista.
- **Installabilità:** semplicità e rapidità con cui il prodotto può essere installato e configurato nell'ambiente di destinazione.
- **Conformità:** aderenza a norme e convenzioni relative alla portabilità, come l'uso di formati standard o l'astrazione dalle specificità del sistema_G operativo.

5.7 Nomenclatura delle Metriche

Per identificare in modo univoco e sistematico tutte le metriche impiegate, viene definita la seguente convenzione:

$$M[\text{Tipo}][\text{Codice}]$$

Dove:

- **M:** prefisso costante che identifica l'elemento come *Metrica*.
- **Tipo:** indica l'ambito di applicazione e assume uno dei seguenti valori:
 - **PC** (Processo): metriche relative alle attività del ciclo di vita e alla gestione dello sviluppo_G.
 - **PD** (Prodotto): metriche relative alle caratteristiche tecniche e qualitative del software realizzato.
- **Codice:** numero progressivo a due cifre (a partire da 00), gestito indipendentemente per ciascun tipo.

5.8 Suddivisione secondo ISO/IEC 12207

In coerenza con lo standard **ISO/IEC 12207**, le metriche sono organizzate in base alla natura dei processi a cui si riferiscono. Tale suddivisione consente di valutare la qualità in modo trasversale, coprendo tanto le attività direttamente produttive quanto quelle di supporto e di governo.

- **Processi primari:** costituiscono il nucleo centrale dello sviluppo_G e includono le attività di fornitura, sviluppo_G e manutenzione del software. Le metriche associate misurano l'efficacia e l'efficienza di queste attività fondamentali.

- **Processi di supporto:** forniscono sostegno ai processi primari durante l'intero ciclo di vita. Rientrano in questa categoria la documentazione, la verifica_G, la gestione della configurazione e l'assicurazione della qualità. Le metriche di supporto aiutano a mantenere un ambiente di lavoro organizzato e controllato.
- **Processi organizzativi:** definiscono la struttura e le regole di funzionamento del gruppo di progetto. Comprendono la gestione dei processi stessi, il miglioramento continuo e la formazione. Le metriche organizzative consentono di monitorare l'efficacia della governance e la crescita del team.

Per quanto riguarda il prodotto, le metriche sono invece raggruppate secondo le sei caratteristiche di qualità descritte in precedenza (funzionalità, affidabilità, efficienza, usabilità, manutenibilità_G, portabilità), così da valutare in modo puntuale il livello qualitativo raggiunto dal software in ciascuna area.

5.8.0.1 Riferimenti

Tutte le definizioni, gli acronimi e le abbreviazioni utilizzati in questo documento sono riportati nel **Glossario**, fornito come documento separato, che garantisce una comprensione uniforme dei termini tecnici e dei concetti rilevanti per il progetto.

6 Metriche di Qualità del Processo

6.1 Processi primari

6.1.1 Fornitura

6.1.1.1 Earned Value (EV)

Sigla	MPC-EV
Formula	$EV_k = BAC \times \text{Percentuale di lavoro completato allo sprint } k$ Legenda: • EV_k: Earned Value allo sprint_G k • BAC: Budget_G at Completion (Budget_G totale previsto) • k: Numero dello sprint_G di riferimento
Descrizione	L'indicatore Earned Value rappresenta il valore del lavoro completato al termine dello sprint _G k rispetto al budget _G totale previsto. Viene calcolato al termine di ciascuno sprint _G per monitorare il progresso reale del progetto rispetto al piano.
Utilità	L'indicatore è utile per monitorare l'andamento del progetto e valutare se il lavoro svolto è in linea con le aspettative.

Tabella 18: Dettagli Metrica MPC-EV - Earned Value

6.1.1.2 Planned Value (PV)

Sigla	MPC-PV
Formula	$PV_k = BAC \times \text{Percentuale di lavoro pianificato allo sprint } k$ Legenda: • PV_k: Planned Value allo sprint_G k • BAC: Budget_G at Completion • k: Numero dello sprint_G di riferimento
Descrizione	L'indicatore Planned Value rappresenta il valore del lavoro pianificato che dovrebbe essere raggiunto al termine dello sprint _G k . Viene calcolato al termine di ciascuno sprint _G per monitorare l'avanzamento del progetto rispetto alla pianificazione.
Utilità	L'indicatore è utile per monitorare l'andamento del progetto e valutare se la pianificazione è rispettata. Il valore pianificato non può essere negativo e deve essere inferiore al BAC.

Tabella 19: Dettagli Metrica MPC-PV - Planned Value

6.1.1.3 Actual Cost (AC)

Sigla	MPC-AC
Formula	$AC_k = \text{Costo effettivo sostenuto nello sprint } k$ Legenda: • AC_k: Actual Cost_G allo sprint_G k
Descrizione	L'indicatore Actual Cost _G rappresenta il costo effettivo sostenuto per completare il lavoro nello sprint _G k .
Utilità	L'indicatore è utile per monitorare l'andamento del progetto e valutare se i costi sono in linea con le aspettative.

Tabella 20: Dettagli Metrica MPC-AC - Actual Cost_G

6.1.1.4 Cost Performance Index (CPI)

Sigla	MPC-CPI
Formula	$CPI_k = \frac{EV_k}{AC_k}$ Legenda: • CPI_k: Cost Performance Index allo sprint_G k • EV_k: Earned Value allo sprint_G k • AC_k: Actual Cost_G allo sprint_G k
Descrizione	Il Cost Performance Index rappresenta il rapporto tra il valore del lavoro completato e il costo effettivo sostenuto, aggiornato allo sprint _G k .
Utilità	Un valore maggiore di 1 indica che il progetto sta rispettando il budget _G , un valore minore di 1 indica che il progetto sta superando il budget _G .

Tabella 21: Dettagli Metrica MPC-CPI - Cost Performance Index

6.1.1.5 Estimate At Completion (EAC)

Sigla	MPC-EAC
Formula	$EAC_k = \frac{BAC}{CPI_k}$ Legenda: • EAC_k: Estimate At Completion calcolato allo sprint_G k • BAC: Budget_G at Completion • CPI_k: Cost Performance Index allo sprint_G k
Descrizione	La metrica Estimate At Completion rappresenta una proiezione del costo finale totale del progetto basata sulla performance attuale rilevata allo sprint _G k .
Utilità	Utilizza il CPI_k come indicatore di efficienza per correggere la stima iniziale (BAC). Se $CPI_k < 1$, EAC_k sarà maggiore del BAC , indicando un probabile sfioramento del budget _G .

Tabella 22: Dettagli Metrica MPC-EAC - Estimate At Completion

6.1.1.6 Estimate To Complete (ETC)

Sigla	MPC-ETC
Formula	$ETC_k = EAC_k - AC_k$ Legenda: • ETC_k: Estimate To Complete allo sprint_G k • EAC_k: Estimate At Completion allo sprint_G k • AC_k: Actual Cost_G allo sprint_G k
Descrizione	La metrica Estimate To Complete stima quanto costerà completare il lavoro rimanente del progetto partendo dallo stato attuale (sprint _G k).
Utilità	Si calcola sottraendo i costi già sostenuti (AC_k) dalla stima del costo finale totale (EAC_k). Utile per la pianificazione del budget _G residuo necessario.

Tabella 23: Dettagli Metrica MPC-ETC - Estimate To Complete

6.1.1.7 Cost Variance (CV)

Sigla	MPC-CV
Formula	$CV_k = EV_k - AC_k$ Legenda: • CV_k: Cost Variance allo sprint_G k • EV_k: Earned Value allo sprint_G k • AC_k: Actual Cost_G allo sprint_G k
Descrizione	Deviazione dal budget _G nello sprint _G corrente. Indica se il lavoro completato è costato più o meno di quanto valga. • $CV > 0$: sotto budget_G (situazione favorevole); • $CV = 0$: in linea con il budget_G; • $CV < 0$: sopra budget_G (situazione sfavorevole).
Unità di misura	Euro (€).

Tabella 24: Dettagli Metrica MPC-CV - Cost Variance

6.1.1.8 Schedule Variance (SV)

Sigla	MPC-SV
Formula	$SV_k = EV_k - PV_k$ Legenda: • SV_k: Schedule Variance allo sprint_G k • EV_k: Earned Value allo sprint_G k • PV_k: Planned Value allo sprint_G k
Descrizione	Deviazione dalla pianificazione nello sprint _G corrente. Indica se il progetto è in anticipo o in ritardo rispetto al piano. • $SV > 0$: in anticipo rispetto al piano; • $SV = 0$: in linea con il piano; • $SV < 0$: in ritardo rispetto al piano.
Unità di misura	Euro (€).

Tabella 25: Dettagli Metrica MPC-SV - Schedule Variance

6.1.1.9 Budget At Completion (BAC)

Sigla	MPC-BAC
Formula	$BAC = 12.860,00 \text{ €}$ Valore fissato alla stipula del contratto, coerente con il <i>Piano di Qualifica_G</i> .
Descrizione	Budget _G totale preventivato per l'intero progetto. Questo valore è fissato alla stipula del contratto e può solo diminuire nel tempo.
Unità di misura	Euro (€).

Tabella 26: Dettagli Metrica MPC-BAC - Budget At Completion

6.2 Processi di supporto

6.2.1 Documentazione

6.2.1.1 Indice di Gulpease (IG)

Sigla	MPC-IG
Formula	$IG = 89 + \frac{300 \cdot N_{frasi} - 10 \cdot N_{lettere}}{N_{parole}}$ Legenda: • IG: Indice di Gulpease • N_{frasi}: Numero di frasi nel testo • $N_{lettere}$: Numero di lettere nel testo • N_{parole}: Numero totale di parole nel testo
Descrizione	Indice che misura la leggibilità di un testo in lingua italiana. Interpretazione: • $IG \geq 80$: testo molto facile, istruzione elementare; • $60 \leq IG < 80$: testo facile, istruzione media; • $40 \leq IG < 60$: testo abbastanza difficile, istruzione superiore; • $IG < 40$: testo difficile, istruzione universitaria. <i>Nota:</i> la metrica viene calcolata esclusivamente in fase di approvazione del documento.
Utilità	Assicura che la documentazione sia comprensibile per il target di riferimento, riducendo ambiguità.

Tabella 27: Dettagli Metrica MPC-IG - Indice di Gulpease

6.2.1.2 Correttezza Ortografica (CO)

Sigla	MPC-CO
Formula	$CO = N_{errori}$ Legenda: • CO: Correttezza Ortografica • N_{errori}: Numero di errori ortografici rilevati
Descrizione	Numero di errori grammaticali o di battitura rilevati nei documenti tramite strumenti automatici di controllo ortografico. <i>Nota:</i> la metrica viene calcolata esclusivamente in fase di approvazione del documento.
Unità di misura	Numero intero (conteggio assoluto).

Tabella 28: Dettagli Metrica MPC-CO - Correttezza Ortografica

6.3 Processi organizzativi

6.3.1 Gestione dei processi

6.3.1.1 Task Completion Rate (TCR)

Sigla	MPC-TCR
Formula	$TCR_k = \frac{T_{tempo}}{T_{tempo} + T_{ritardo}} \times 100$ Legenda: • TCR_k: Task Completion Rate allo sprint_G k • T_{tempo}: Numero di task completati in tempo • $T_{ritardo}$: Numero di task completati in ritardo
Descrizione	Percentuale di task completati entro la scadenza dello sprint _G rispetto al totale dei task chiusi nello sprint _G (in tempo e in ritardo). Misura la capacità del team di rispettare le scadenze prefissate. Un task è considerato “completato in tempo” se la sua End Date è precedente o uguale alla Target Date dello sprint _G .
Unità di misura	Percentuale (%).

Tabella 29: Dettagli Metrica MPC-TCR - Task Completion Rate

6.3.1.2 Task Slippage (TS)

Sigla	MPC-TS
Formula	$TS_k = \frac{T_{posticipati}}{T_{totali}} \times 100$ Legenda: • TS_k: Task Slippage allo sprint_G k • $T_{posticipati}$: Numero di task posticipati allo sprint_G successivo • T_{totali}: Numero totale di task pianificati per lo sprint_G
Descrizione	Percentuale di task pianificati per lo sprint _G corrente che non sono stati portati a termine entro la sua conclusione e vengono posticipati allo sprint _G successivo. Un task è considerato “posticipato” se alla chiusura dello sprint _G il suo stato è diverso da “Done”.
Unità di misura	Percentuale (%).

Tabella 30: Dettagli Metrica MPC-TS - Task Slippage

6.3.1.3 Merge Rework (MR)

Sigla	MPC-MR
Formula	$MR_k = \frac{L_{rilavorato}}{L_{totale}} \times 100$ Legenda: • MR_k: Merge Rework allo sprint_G k • $L_{rilavorato}$: Unità o ore di lavoro rilavorato nello sprint_G k • L_{totale}: Lavoro totale prodotto nello sprint_G k
Descrizione	La metrica Merge Rework misura la percentuale di lavoro che è stato necessario ripetere rispetto al lavoro totale svolto. Indica l'efficienza del processo produttivo e la qualità delle attività a monte. • $MR = 0\%$: nessuna rilavorazione (situazione ottimale); • MR basso: rilavorazione contenuta (situazione accettabile); • MR alto: elevata rilavorazione (situazione sfavorevole, da investigare).
Unità di misura	Percentuale (%).

Tabella 31: Dettagli Metrica MPC-MR - Merge Rework

6.3.1.4 Issue Reopening Rate (IR)

Sigla	MPC-IR
Formula	$IR_k = \frac{I_{riaperti}}{I_{risolti}} \times 100$ Legenda: • IR_k: Issue Reopening Rate allo sprint_G k • $I_{riaperti}$: Numero di problemi riaperti nello sprint_G k • $I_{risolti}$: Numero totale di problemi risolti nello sprint_G k
Descrizione	La metrica Issue Reopening Rate misura la percentuale di problemi che, dopo essere stati contrassegnati come risolti, vengono riaperti. Un valore elevato può indicare carenze nel processo di verifica_G o nella qualità delle correzioni applicate. • $IR \leq 15\%$: valore accettabile; • $IR > 15\%$: valore critico, necessario intervento correttivo.
Unità di misura	Percentuale (%).

Tabella 32: Dettagli Metrica MPC-IR - Issue Reopening Rate

6.3.1.5 Percentuale Metriche Soddisfatte (PMS)

Sigla	MPC-PMS
Formula	$PMS_k = \frac{M_{soddisfatte}}{M_{totali}} \times 100$ Legenda: • PMS_k: Percentuale Metriche Soddisfatte allo sprint_G k • $M_{soddisfatte}$: Numero di metriche che rientrano nei valori accettabili allo sprint_G k • M_{totali}: Numero totale di metriche misurate allo sprint_G k
Descrizione	La metrica Percentuale Metriche Soddisfatte misura la quota di metriche di qualità che rientrano nei valori accettabili definiti nel Piano di Qualifica_G. Dal calcolo sono escluse le metriche di correttezza dei documenti, in quanto calcolate a documento approvato e non a sprint_G, il cui coinvolgimento produrrebbe uno sfasamento che non permetterebbe una stima affidabile. • $PMS \geq 80\%$: valore accettabile; • $PMS < 80\%$: valore critico, necessario intervento correttivo.
Unità di misura	Percentuale (%).

Tabella 33: Dettagli Metrica MPC-PMS - Percentuale Metriche Soddisfatte

7 Metriche di Qualità del Prodotto

Le metriche di qualità del prodotto misurano le caratteristiche del software realizzato: valutano direttamente il comportamento dell'applicazione, la sua affidabilità, la sua facilità d'uso e la sua capacità di soddisfare i requisiti_G del capitolato_G. Sono identificate con il prefisso **MPD**. In questo documento se ne fornisce la definizione (descrizione, formula, unità di misura ed eventuale strumento di rilevazione); le soglie di accettabilità e i valori ottimali sono definiti nel Piano di Qualifica_G.

7.1 Funzionalità

7.1.1 Copertura Requisiti Obbligatori (MPD-CRO)

Sigla	MPD-CRO
Formula	$CRO = \frac{RO_{soddisfatti}}{RO_{totali}} \times 100$
Descrizione	Percentuale di requisiti_G obbligatori correttamente implementati e verificati rispetto al totale dei requisiti_G obbligatori individuati nell'Analisi dei Requisiti_G. Un requisito_G è considerato "soddisfatto" quando il relativo test_G di sistema_G risulta superato.
Unità di misura	Percentuale (%).

Tabella 34: Dettagli Metrica MPD-CRO - Copertura Requisiti Obbligatori

7.1.2 Copertura Requisiti Desiderabili (MPD-CRD)

Sigla	MPD-CRD
Formula	$CRD = \frac{RD_{soddisfatti}}{RD_{totali}} \times 100$
Descrizione	Percentuale di requisiti _G desiderabili correttamente implementati e verificati rispetto al totale dei requisiti _G desiderabili individuati.
Unità di misura	Percentuale (%).

Tabella 35: Dettagli Metrica MPD-CRD - Copertura Requisiti Desiderabili

7.1.3 Copertura Requisiti Opzionali (MPD-CROp)

Sigla	MPD-CROp
Formula	$CROp = \frac{ROp_{soddisfatti}}{ROp_{totali}} \times 100$
Descrizione	Percentuale di requisiti _G opzionali correttamente implementati e verificati rispetto al totale dei requisiti _G opzionali individuati.
Unità di misura	Percentuale (%).

Tabella 36: Dettagli Metrica MPD-CROp - Copertura Requisiti Opzionali

7.1.4 Copertura Funzioni LLM Obbligatorie (MPD-CLLM)

Sigla	MPD-CLLM
Formula	$CLLM = \frac{F_{implementate}}{F_{obbligatorie}} \times 100$
Descrizione	Percentuale di funzioni basate sull'LLM _G obbligatorie (es. generazione, riassunto, traduzione) correttamente implementate e verificate rispetto al totale di quelle richieste dal capitolato _G .
Unità di misura	Percentuale (%).

Tabella 37: Dettagli Metrica MPD-CLLM - Copertura Funzioni LLM Obbligatorie

7.1.5 Correttezza Rendering Markdown (MPD-CMD)

Sigla	MPD-CMD
Formula	$CMD = \frac{C_{corretti}}{C_{previsti}} \times 100$
Descrizione	Percentuale di casi di rendering Markdown _G resi correttamente rispetto ai casi previsti (intestazioni, elenchi, stili, blocchi di codice, tabelle, formule, ecc.).
Unità di misura	Percentuale (%).

Tabella 38: Dettagli Metrica MPD-CMD - Correttezza Rendering Markdown

7.1.6 Salvataggio e Recupero Note (MPD-SN)

Sigla	MPD-SN
Formula	$SN = \frac{C_{superati}}{C_{obbligatori}} \times 100$
Descrizione	Percentuale di casi obbligatori di salvataggio e successivo recupero delle note superati con successo rispetto al totale dei casi obbligatori previsti, senza perdita o alterazione del contenuto.
Unità di misura	Percentuale (%).

Tabella 39: Dettagli Metrica MPD-SN - Salvataggio e Recupero Note

7.2 Affidabilità

7.2.1 Data Loss (MPD-DL)

Sigla	MPD-DL
Formula	$DL = N_{perdite}$
Descrizione	Numero di occorrenze in cui, durante i test _G di salvataggio e caricamento, il contenuto di una nota risulta perso o alterato. Misura l'affidabilità della catena di persistenza dei dati.
Unità di misura	Numero intero (conteggio assoluto).

Tabella 40: Dettagli Metrica MPD-DL - Data Loss

7.2.2 Error Handling (MPD-EH)

Sigla	MPD-EH
Formula	$EH = \frac{E_{gestiti}}{E_{totali}} \times 100$
Descrizione	Percentuale di condizioni di errore (errori dell'LLM _G , di rete, di input _G non valido) gestite correttamente dal sistema _G rispetto al totale delle condizioni di errore sottoposte a test _G .
Unità di misura	Percentuale (%).

Tabella 41: Dettagli Metrica MPD-EH - Error Handling

7.2.3 Session Recovery (MPD-SR)

Sigla	MPD-SR
Formula	$SR = \frac{R_{riusciti}}{R_{previsti}} \times 100$
Descrizione	Percentuale di casi previsti di ripristino della sessione (recupero del testo non salvato dopo un'interruzione o una chiusura imprevista) andati a buon fine rispetto al totale dei casi previsti.
Unità di misura	Percentuale (%).

Tabella 42: Dettagli Metrica MPD-SR - Session Recovery

7.2.4 Availability Rate (MPD-AR)

Sigla	MPD-AR
Formula	$AR = \frac{T_{disponibile}}{T_{osservato}} \times 100$
Descrizione	Percentuale di tempo in cui l'applicazione risulta disponibile e operativa durante le sessioni di test _G rispetto al tempo totale osservato.
Unità di misura	Percentuale (%).

Tabella 43: Dettagli Metrica MPD-AR - Availability Rate

7.3 Usabilità

7.3.1 Learning Time (MPD-LT)

Sigla	MPD-LT
Formula	$LT = \frac{\sum_{i=1}^n T_i}{n}$
Descrizione	Tempo medio necessario a un utente per completare con successo una funzionalità per la prima volta, senza assistenza esterna; T_i è il tempo impiegato dall' i -esimo utente e n il numero di utenti coinvolti nella sessione di test _G di usabilità. Misura l'apprendibilità dell'interfaccia.
Unità di misura	Minuti.

Tabella 44: Dettagli Metrica MPD-LT - Learning Time

7.3.2 Navigation Errors (MPD-NE)

Sigla	MPD-NE
Formula	$NE = \frac{\sum_{i=1}^n E_i}{n}$
Descrizione	Numero medio di errori di navigazione (azioni errate o percorsi sbagliati) commessi dagli utenti per scenario di test _G ; E_i è il numero di errori nell' i -esimo scenario e n il numero di scenari valutati.
Unità di misura	Numero di errori per scenario.

Tabella 45: Dettagli Metrica MPD-NE - Navigation Errors

7.3.3 UI Consistency (MPD-UI)

Sigla	MPD-UI
Formula	$UI = \frac{El_{coerenti}}{El_{totali}} \times 100$
Descrizione	Percentuale di elementi dell'interfaccia coerenti con le convenzioni grafiche e comportamentali definite rispetto al totale degli elementi valutati.
Unità di misura	Percentuale (%).

Tabella 46: Dettagli Metrica MPD-UI - UI Consistency

7.3.4 Feature Understandability (MPD-FU)

Sigla	MPD-FU
Formula	$FU = \frac{F_{comprese}}{F_{totali}} \times 100$
Descrizione	Percentuale di funzioni che l'utente comprende e utilizza correttamente senza ricorrere a una guida rispetto al totale delle funzioni valutate.
Unità di misura	Percentuale (%).

Tabella 47: Dettagli Metrica MPD-FU - Feature Understandability

7.4 Efficienza

7.4.1 Response Time Interfaccia (MPD-RT)

Sigla	MPD-RT
Formula	$RT = \frac{\sum_{i=1}^n (T_{risposta,i} - T_{richiesta,i})}{n}$
Descrizione	Tempo medio di risposta dell'interfaccia locale a un input _G dell'utente, misurato come intervallo tra l'invio della richiesta e la ricezione della risposta completa.
Unità di misura	Secondi (s).
Strumento di rilevazione	Test _G automatici di performance.

Tabella 48: Dettagli Metrica MPD-RT - Response Time Interfaccia

7.4.2 Markdown Rendering Time (MPD-MRT)

Sigla	MPD-MRT
Formula	$MRT = \frac{\sum_{i=1}^n T_i}{n}$
Descrizione	Tempo medio necessario a rendere in anteprima il testo Markdown _G dopo una modifica; T_i è il tempo di rendering della i -esima misurazione.
Unità di misura	Secondi (s).
Strumento di rilevazione	Test _G automatici di performance.

Tabella 49: Dettagli Metrica MPD-MRT - Markdown Rendering Time

7.4.3 LLM Response Time (MPD-LLMRT)

Sigla	MPD-LLMRT
Formula	$LLMRT = \frac{\sum_{i=1}^n T_i}{n}$
Descrizione	Tempo medio tra l'invio di una richiesta al servizio LLM _G e la ricezione della risposta completa; T_i è il tempo dell' i -esima richiesta.
Unità di misura	Secondi (s).
Strumento di rilevazione	Test _G automatici di performance.

Tabella 50: Dettagli Metrica MPD-LLMRT - LLM Response Time

7.4.4 Loading Speed (MPD-LS)

Sigla	MPD-LS
Formula	$LS = T_{caricamento}$
Descrizione	Tempo necessario al caricamento iniziale completo dell'applicazione nel browser, dal primo accesso fino a quando l'interfaccia è pronta all'uso.
Unità di misura	Secondi (s).
Strumento di rilevazione	Test _G automatici di performance.

Tabella 51: Dettagli Metrica MPD-LS - Loading Speed

7.5 Manutenibilità

7.5.1 Complessità Ciclomatica (MPD-CYC)

Sigla	MPD-CYC
Formula	$M = E - N + 2P$
Descrizione	Numero di percorsi linearmente indipendenti nel flusso di controllo, dove E è il numero di archi, N il numero di nodi e P il numero di componenti connesse (solitamente 1 per singola funzione). Il valore riportato è la media della complessità ciclomatica su tutte le funzioni del sistema _G .
Unità di misura	Numero intero (adimensionale).
Strumento di rilevazione	Analizzatore statico del codice (es. <code>gocyclo</code> per Go, ESLint per TypeScript).

Tabella 52: Dettagli Metrica MPD-CYC - Complessità Ciclomatica

7.5.2 Code Smell (MPD-CS)

Sigla	MPD-CS
Formula	$CS = \frac{N_{smell}}{N_{componenti}}$
Descrizione	Numero medio di code smell (indicatori di progettazione debole, come metodi troppo lunghi, classi troppo grandi o duplicazioni) rilevati per componente del sistema _G .
Unità di misura	Numero di code smell per componente.
Strumento di rilevazione	Analizzatore statico del codice (es. SonarQube, ESLint, <code>golangci-lint</code>).

Tabella 53: Dettagli Metrica MPD-CS - Code Smell

7.5.3 Duplicazione del Codice (MPD-DUP)

Sigla	MPD-DUP
Formula	$DUP = \frac{L_{duplicate}}{L_{totali}} \times 100$
Descrizione	Percentuale di righe di codice duplicate rispetto al totale delle righe di codice del sistema _G . Una duplicazione elevata riduce la manutenibilità _G .
Unità di misura	Percentuale (%).
Strumento di rilevazione	Analizzatore statico del codice (es. SonarQube).

Tabella 54: Dettagli Metrica MPD-DUP - Duplicazione del Codice

7.5.4 Modularità (MPD-MOD)

Sigla	MPD-MOD
Formula	$MOD = \frac{D_{effettive}}{n \times (n-1)}$
Descrizione	Grado di accoppiamento tra i moduli del sistema _G , dove $D_{effettive}$ è il numero di dipendenze direzionali effettive e $n \times (n-1)$ il numero massimo di dipendenze possibili tra n moduli. Valori più bassi indicano moduli più indipendenti e quindi una maggiore modularità.
Unità di misura	Adimensionale (scala 0-1).
Strumento di rilevazione	Analisi delle dipendenze tramite strumenti di analisi statica.

Tabella 55: Dettagli Metrica MPD-MOD - Modularità

7.6 Sicurezza

7.6.1 Secret Key Exposure (MPD-SK)

Sigla	MPD-SK
Formula	$SK = N_{chiavi\ esposte}$
Descrizione	Numero di chiavi di accesso, credenziali o segreti esposti in chiaro nel codice sorgente, nei log o nei file di configurazione versionati.
Unità di misura	Numero intero (conteggio assoluto).
Strumento di rilevazione	Strumenti di <i>secret scanning</i> e analisi statica.

Tabella 56: Dettagli Metrica MPD-SK - Secret Key Exposure

7.6.2 Input Validation (MPD-IV)

Sigla	MPD-IV
Formula	$IV = \frac{I_{validati}}{I_{totali}} \times 100$
Descrizione	Percentuale di punti di ingresso dei dati (input _G dell'utente, parametri delle chiamate API _G) sottoposti a validazione rispetto al totale dei punti di ingresso individuati.
Unità di misura	Percentuale (%).

Tabella 57: Dettagli Metrica MPD-IV - Input Validation